

《计算机系统基础》

第3章 程序的机器级表示 III：过程

授课老师:

何璐璐

luluhe@whu.edu.cn

前两节内容——程序的机器级表示：基础&控制

- 英特尔处理器和架构的历史
- 汇编基础：寄存器、操作数、传送
- 算术和逻辑运算
- C、汇编、机器代码
- 控制：条件码
- 条件分支

条件码和条件分支

■ 单比特位条件码寄存器

- | | | | |
|------|-----------|----|-----------|
| ■ CF | 进位标志（无符号） | SF | 符号标志（有符号） |
| ■ ZF | 零标志 | OF | 溢出标志（有符号） |

■ 算术运算的隐式设置

示例: `addq Src, Dest` $\leftrightarrow$ `t = a+b`

■ 通过 `compare` 指令来显式设置条件码

- `cmpq b, a` 就像执行 `a-b` 但不改变目的寄存器的值

■ 通过 `test` 指令进行显式设置

- `testq b, a` // `testq b, a` 类似于执行 `a&b` 但不改变目的寄存器的值

■ 通过 `setX` 指令来显示读取条件码

- `setX Dest` // 根据条件码不同组合的结果设置目的寄存器的最低字节为0或1。不改变操作数 `Dest` 的剩余 7 个字节。

■ `jX` 指令：根据条件代码跳转到代码的不同部分，隐式读取条件代码

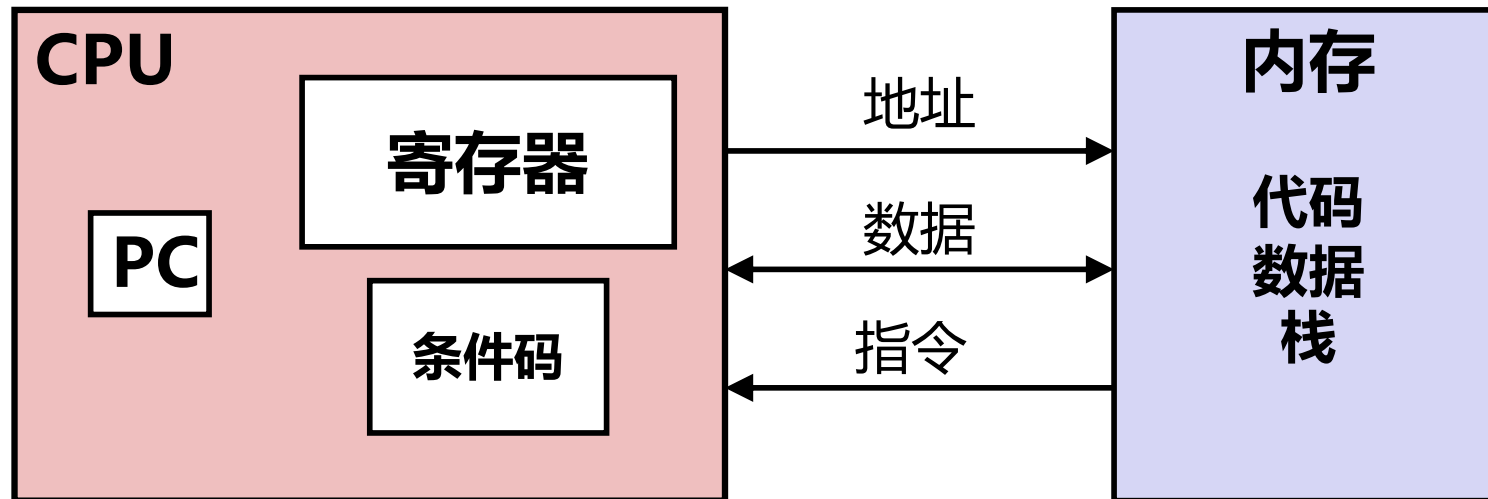
■ `cmovX` 指令：当传送条件满足时，将源操作数的值S传送到目的寄存器或内存

本节内容

■ 过程

- 栈的结构
- 调用的惯例
 - 控制转移
 - 数据传送
 - 管理局部数据
- 递归

汇编/机器代码视图



一个程序员应该知道：

■ PC: 程序计数器

- 下一条指令的地址
- 被叫做“RIP” (x86-64)

■ 寄存器文件

- 程序频繁使用的数据

■ 条件码

- 储存最近一次算术或逻辑运算的结果
- 用于条件分支语句

■ 内存

- 字节可寻址数组
- 代码和用户数据
- 为程序创建栈帧

过程调用的机制

■ 控制转移

- 进入到过程代码的开始
- 返回到返回点

■ 数据传送

- 一个或多个参数
- 返回值

■ 内存管理

- 在程序执行期间分配
- 在返回前进行释放

■ 这些机制全部由机器指令实现

■ x86-64 的过程实现仅使用了必须的机制

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```

过程调用的机制

■ 控制转移

- 进入到过程代码的开始
- 返回到返回点

■ 数据传送

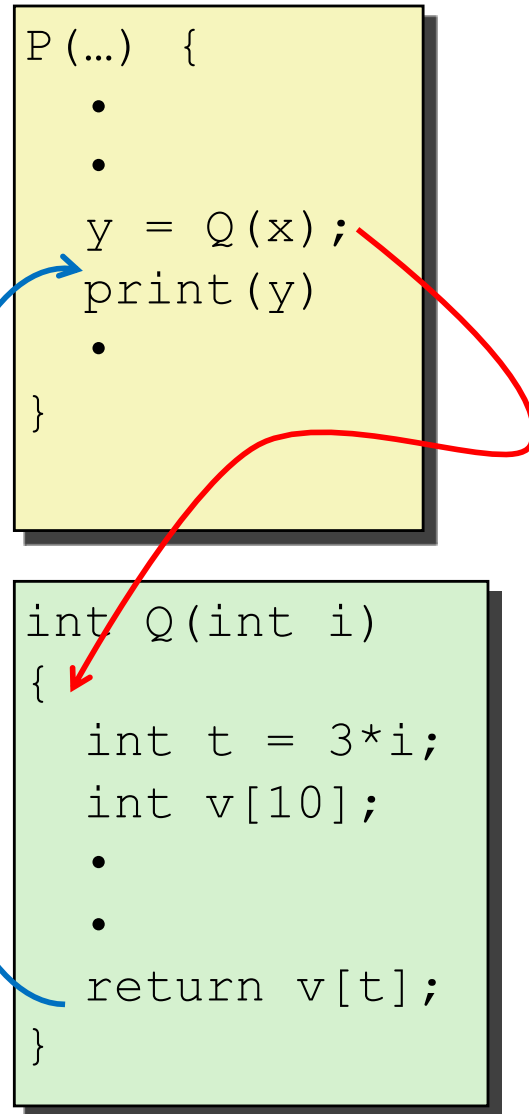
- 一个或多个参数
- 返回值

■ 内存管理

- 在程序执行期间分配
- 在返回前进行释放

■ 这些机制全部由机器指令实现

■ x86-64 的过程实现仅使用了必须的机制



过程调用的机制

■ 控制转移

- 进入到过程代码的开始
- 返回到返回点

■ 数据传送

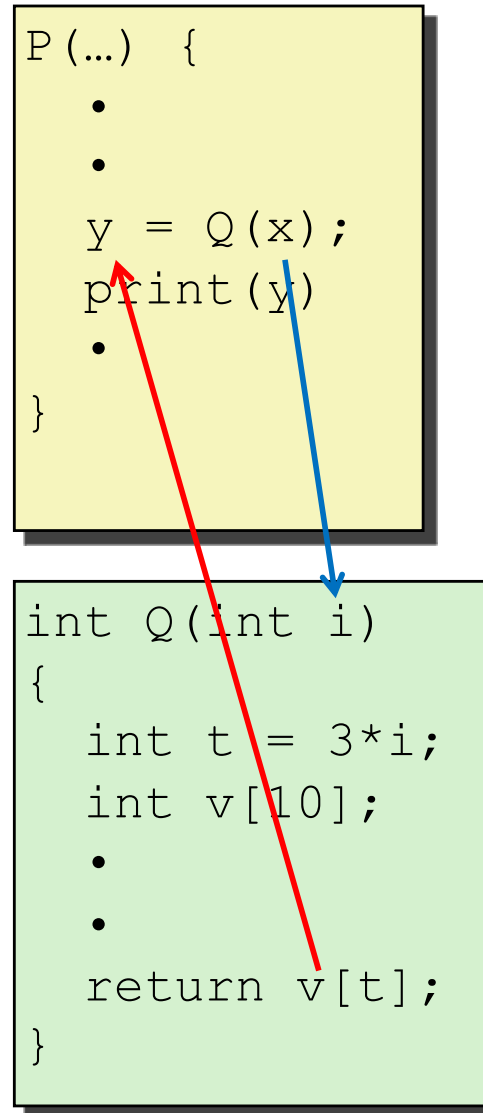
- 一个或多个参数
- 返回值

■ 内存管理

- 在程序执行期间分配
- 在返回前进行释放

■ 这些机制全部由机器指令实现

■ x86-64 的过程实现仅使用了必须的机制



过程调用的机制

■ 控制转移

- 进入到过程代码的开始
- 返回到返回点

■ 数据传送

- 一个或多个参数
- 返回值

■ 内存管理

- 在程序执行期间分配
- 在返回前进行释放

■ 这些机制全部由机器指令实现

■ x86-64 的过程实现仅使用了必须的机制

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```

过程调用的机制

■ 控制转移

- 进入到过程代码的开始
- 返回到返回点

■ 传递数据

- 一个或多个参数
- 返回值

■ 内存管理

- 在程序执行期间分配
- 在返回前进行释放

■ 这些机制全部由机器指令实现

■ x86-64 的过程实现仅使用了必须的机制

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```

本节内容

■ 过程

■ 栈的结构

■ 调用的惯例

■ 控制转移

■ 数据传送

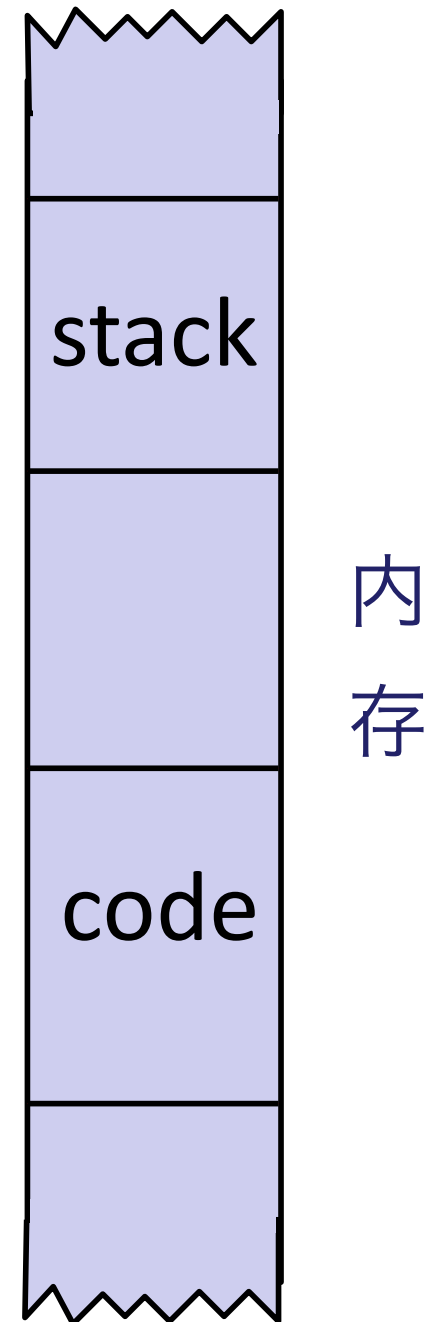
■ 管理局部数据

■ 递归

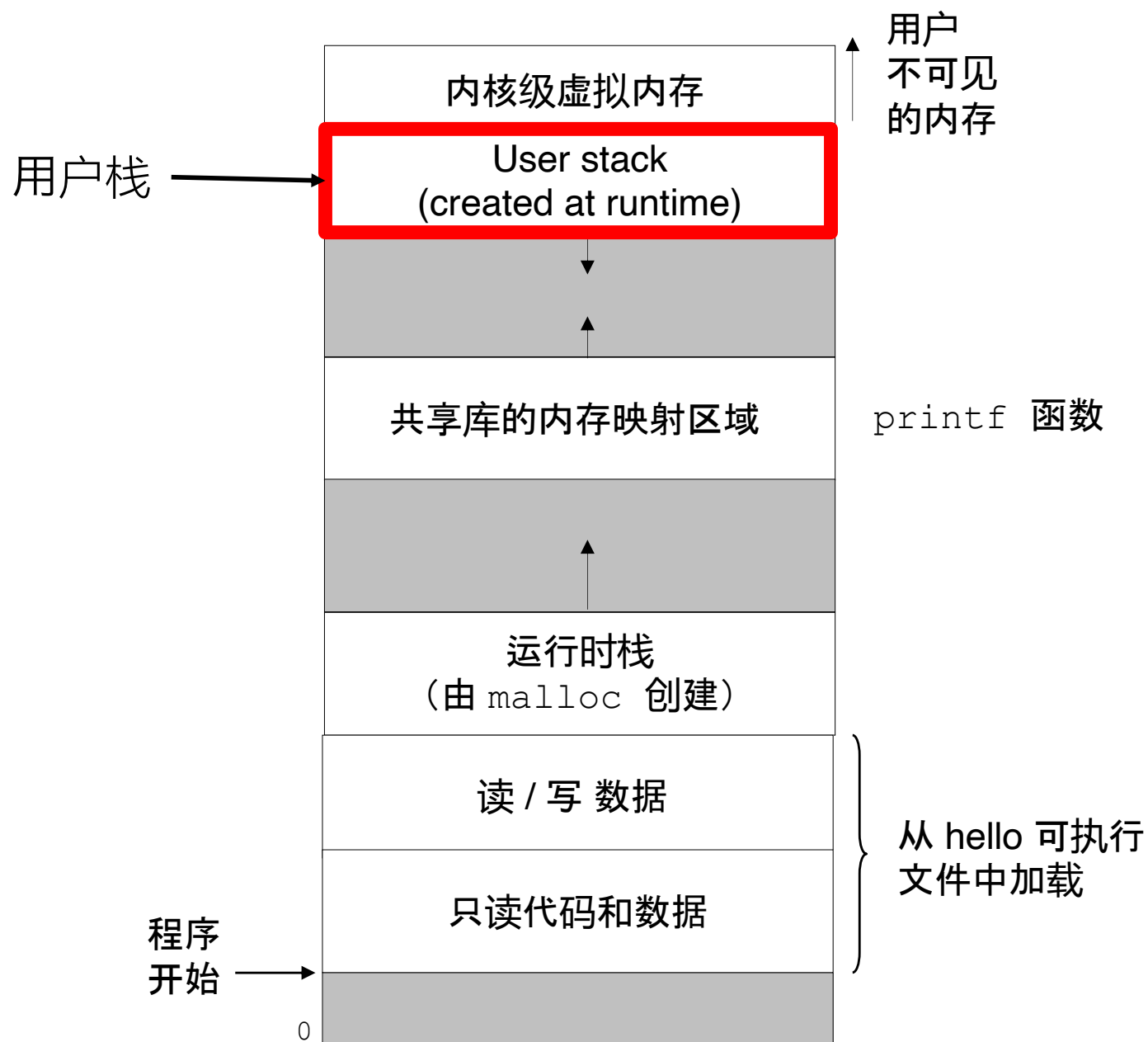
x86-64 的栈

■ 使用栈的规则管理的内存

- 内存被视为字节数组。
- 不同的区域有不同的目的。

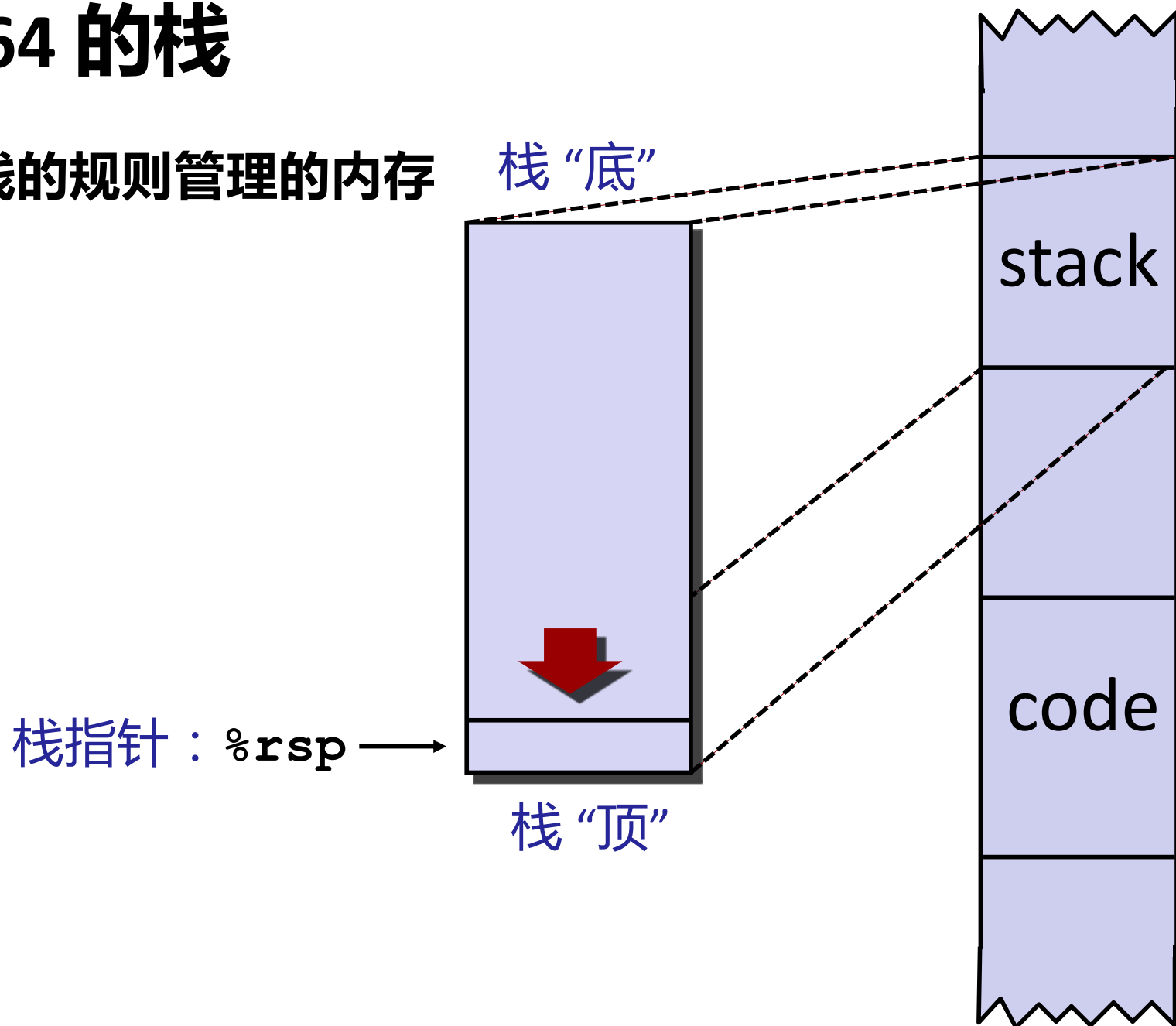


进程虚拟地址空间



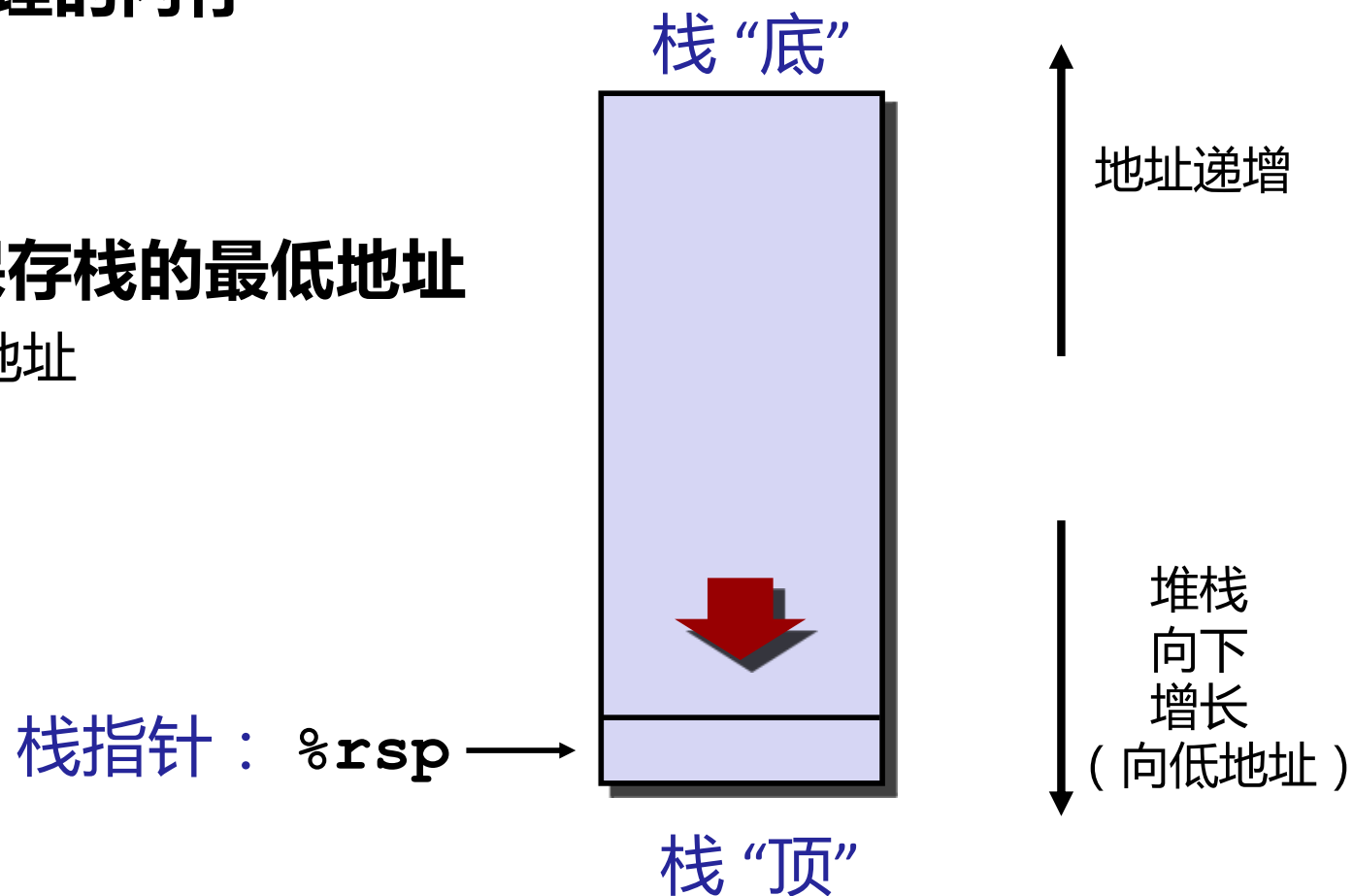
x86-64 的栈

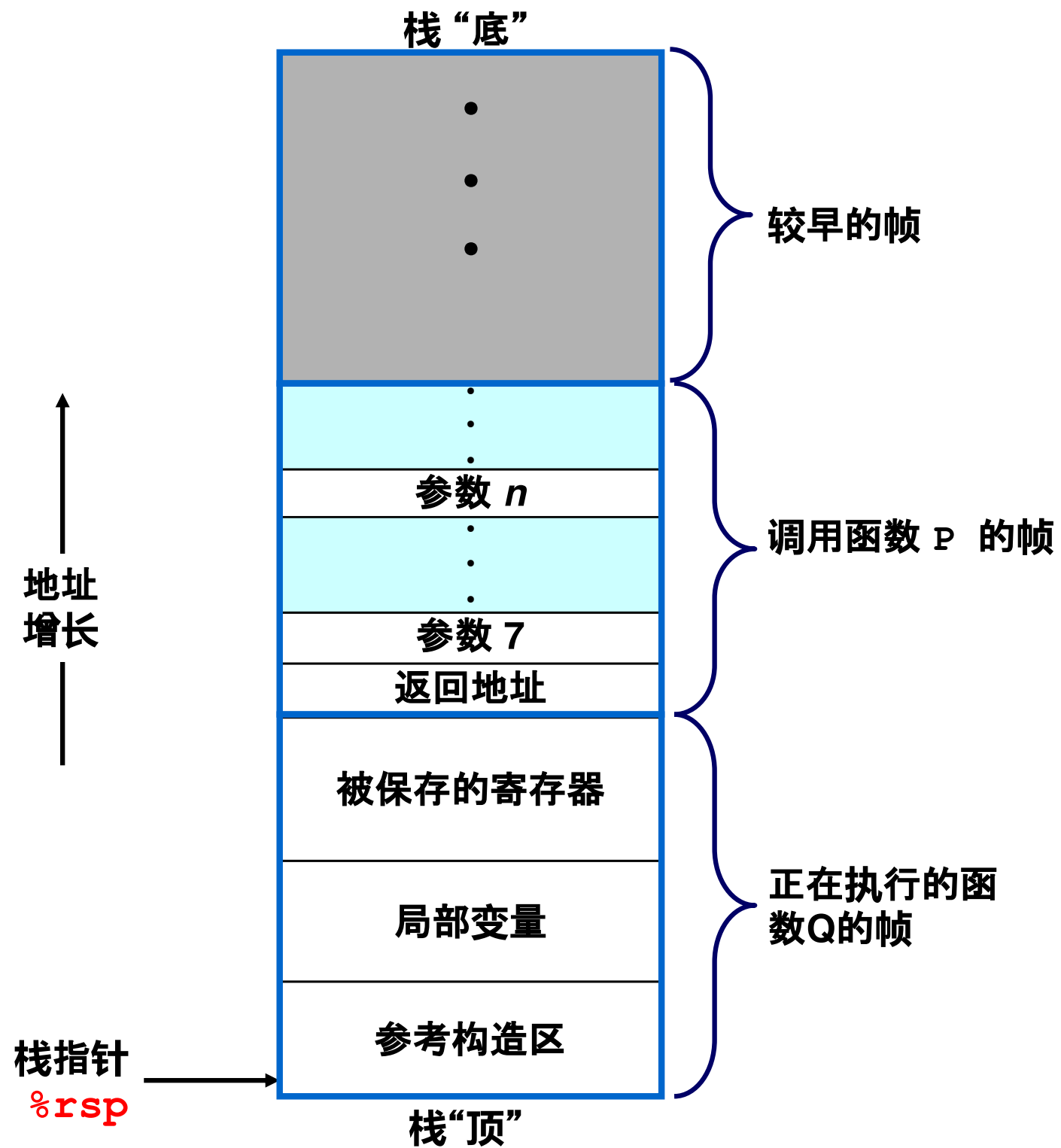
■ 使用栈的规则管理的内存



x86-64 的栈

- 使用栈的规则管理的内存
- 向低地址增长
- 寄存器 `%rsp` 保存栈的最低地址
 - “栈顶”元素的地址

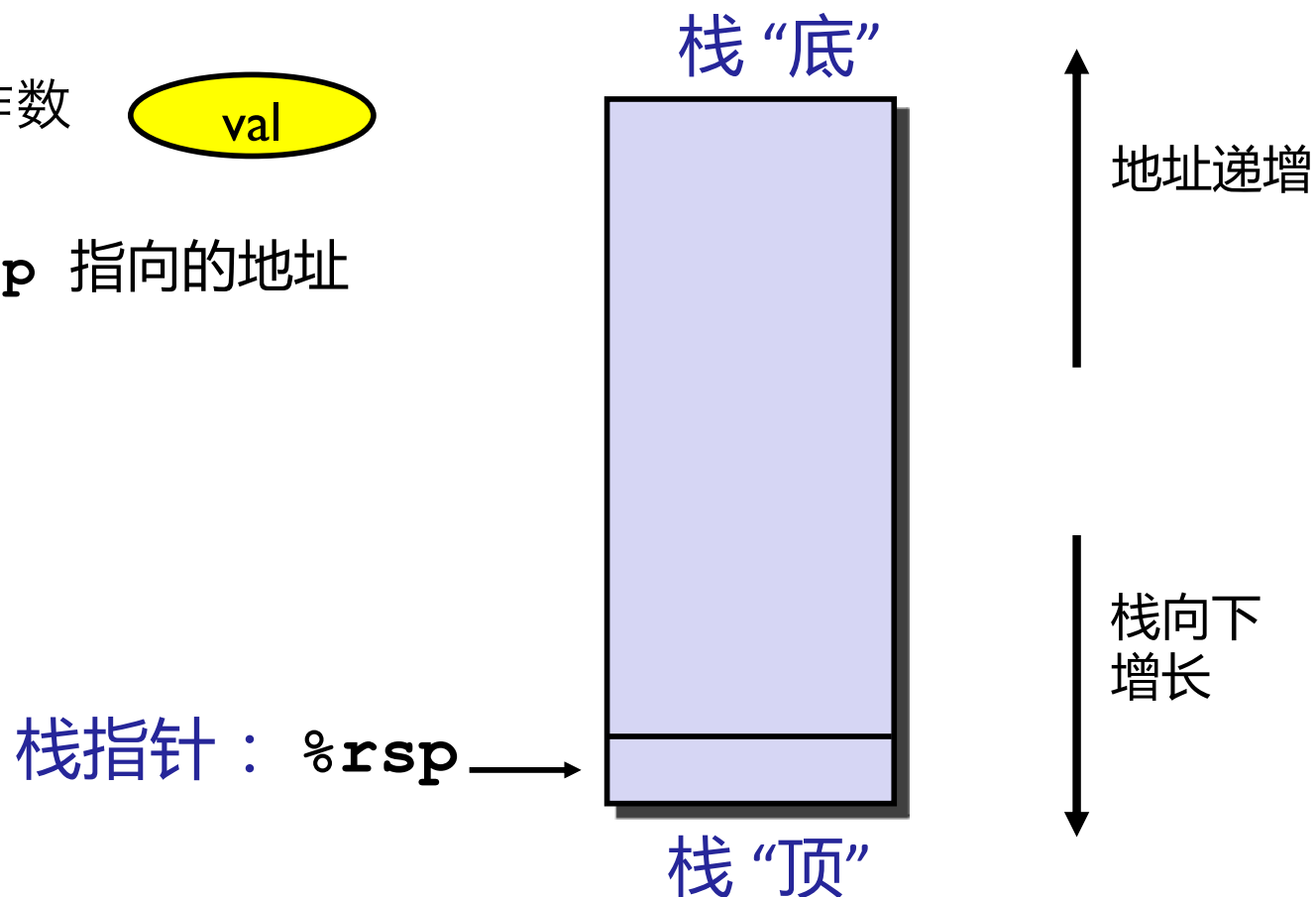




x86-64 的栈：Push

■ pushq Src

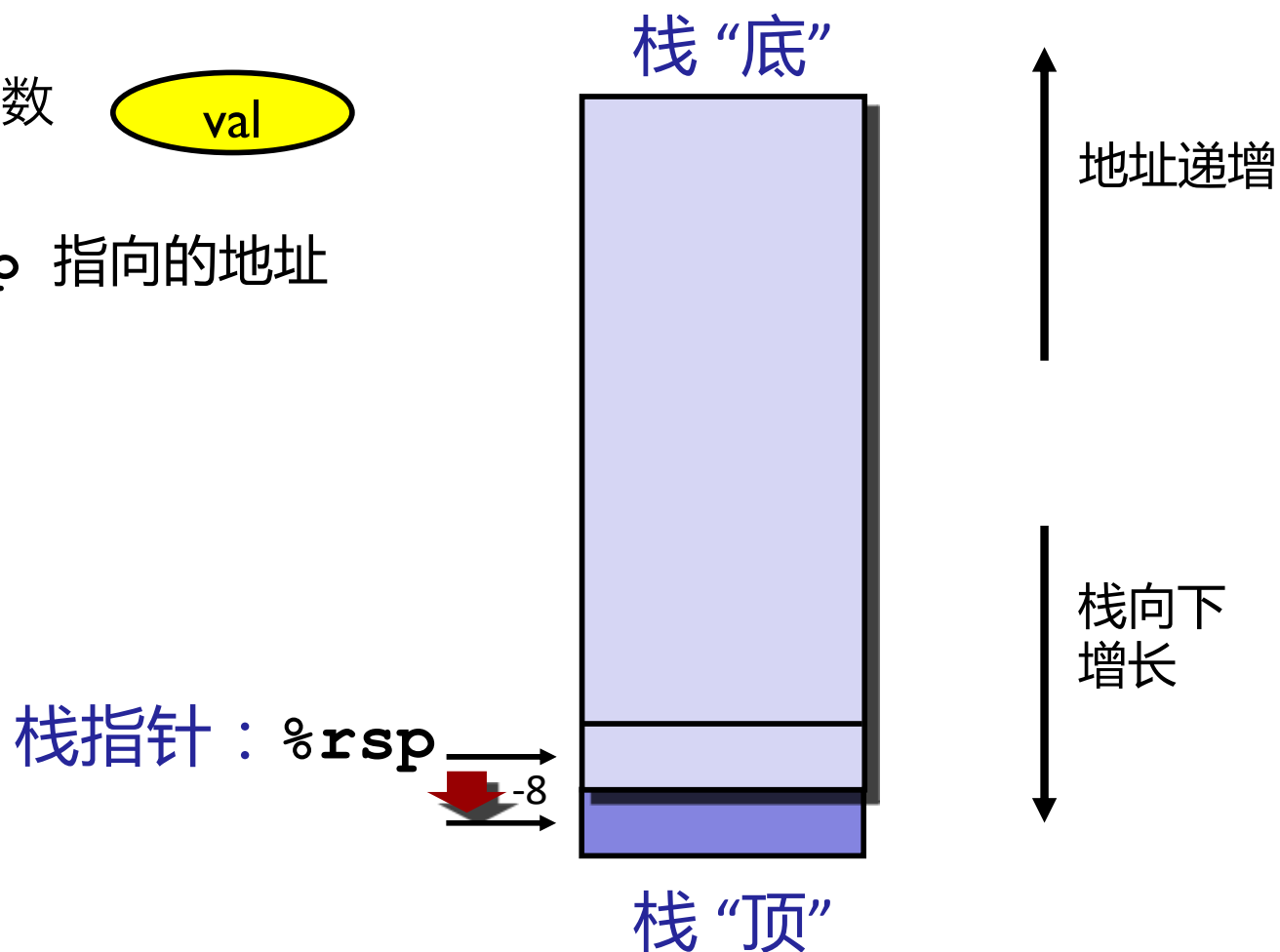
- 在 **Src** 处获取操作数 val
- **%rsp** 的值减 8
- 将操作数写入 **%rsp** 指向的地址



x86-64 的栈：Push

■ pushq Src

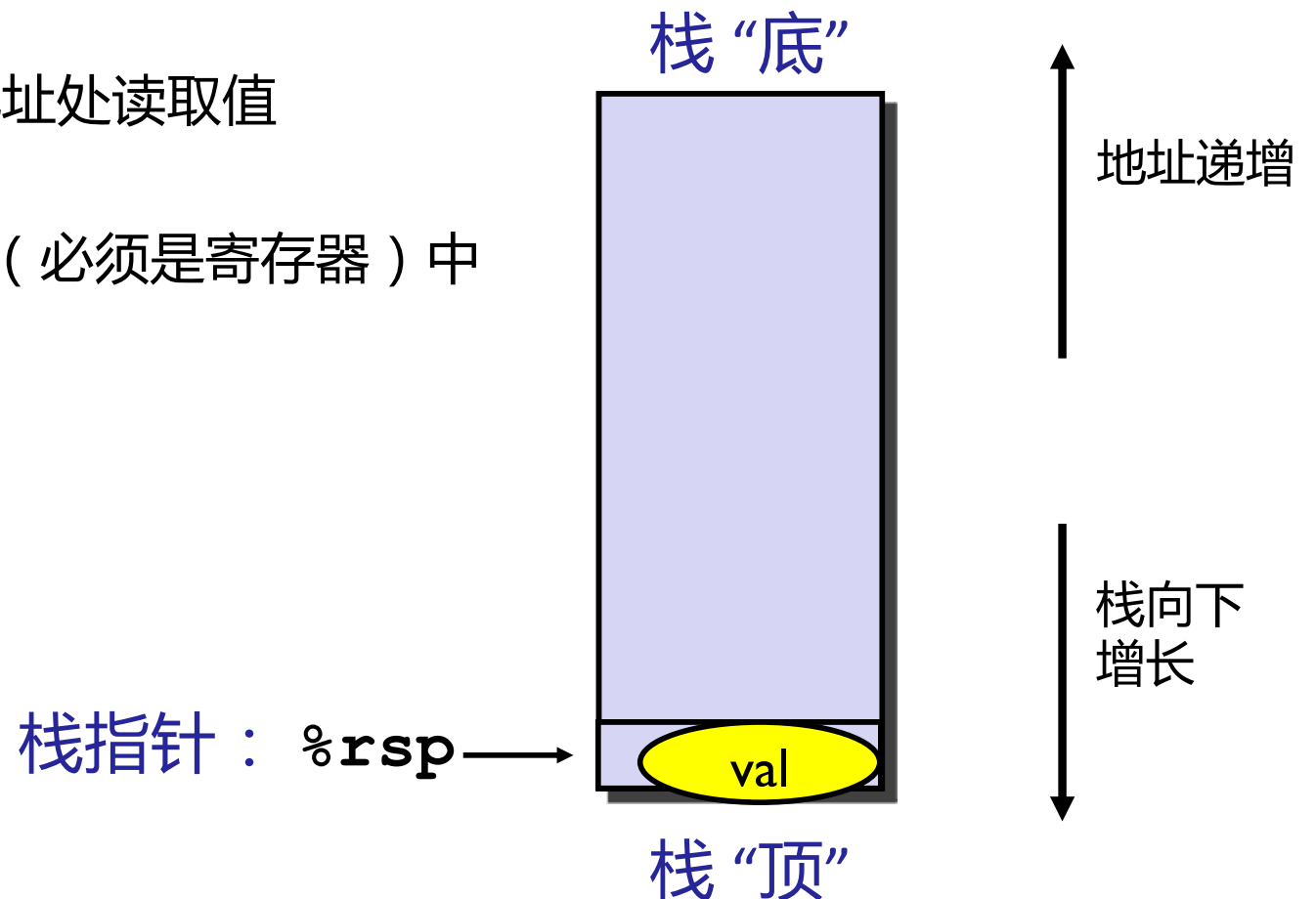
- 在 **Src** 处获取操作数 val
- **%rsp** 的值减 8
- 将操作数写入 **%rsp** 指向的地址



x86-64 的栈：Pop

■ popq Dest

- 从`%rsp` 指向的地址处读取值
- `%rsp` 的值加8
- 将值存储在 `Dest` (必须是寄存器) 中



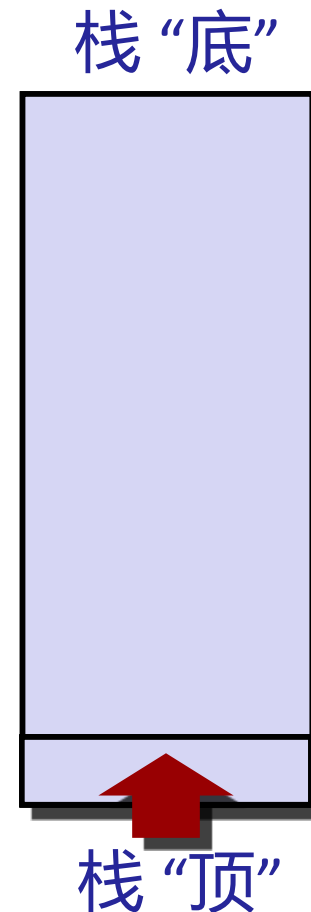
x86-64 的栈：Pop

■ popq Dest

- 从`%rsp` 指向的地址处读取值
- `%rsp` 的值加8
- 将值存储在 `Dest` (必须是寄存器) 中

val

栈指针：`%rsp`  +8



地址递增

栈向下
增长

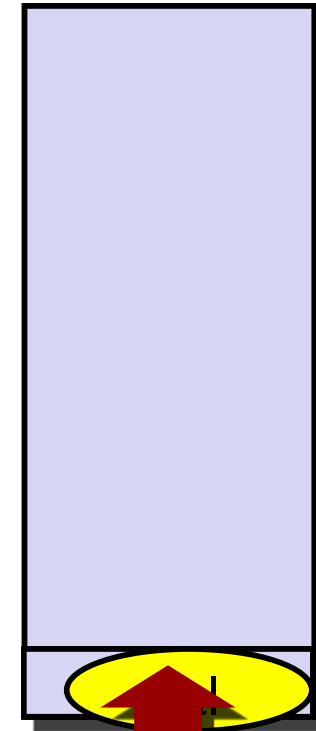
x86-64 的栈：Pop

■ popq Dest

- 从`%rsp` 指向的地址处读取值
- `%rsp` 的值加8
- 将值存储在 `Dest` (必须是寄存器) 中

栈指针：`%rsp` →

栈“底”



栈“顶”

地址递增

栈向下
增长

(内存没有变化，只有`%rsp`的值在改变)

本节内容

■ 过程

- 栈的结构
- 调用的惯例
 - 控制转移
 - 数据传送
 - 管理局部数据
- 递归

过程调用代码示例

```
void multstore(long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

```
00000000000400540 <multstore>:
400540: push    %rbx                # Save %rbx
400541: mov     %rdx,%rbx           # Save dest
400544: callq   400550 <mult2>      # mult2(x,y)
400549: mov     %rax, (%rbx)         # Save at dest
40054c: pop     %rbx                # Restore %rbx
40054d: retq                      # Return
```

```
long mult2(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
00000000000400550 <mult2>:
400550: mov     %rdi,%rax           # a
400553: imul    %rsi,%rax           # a * b
400557: retq                        # Return
```

过程的控制流

■ 用栈支持过程调用和返回

■ 过程调用：`call label`

- 将返回地址压入栈中
- 跳转到 `label` 处

■ 返回地址:

- 调用后下一条指令的地址
- 例如反汇编

■ 过程返回：`ret`

- 从栈中弹出地址
- 跳转到该地址

控制流示例 #1

0000000000400540 <multstore>:

•
•
•
•
•

400544: callq 400550 <mult2>

400549: mov %rax, (%rbx)

0000000000400550 <mult2>:

400550: mov %rdi, %rax

•
•

400557: retq

0x130

0x128

0x120

%rsp

0x120

%rip

0x400544

控制流示例 #2

```
00000000000400540 <multstore>:
```

•
•
•
•
•

```
400544: callq 400550 <mult2>
```

```
400549: mov    %rax, (%rbx) ←
```

0x130

0x128

0x120

0x118

0x400549

%rsp

0x118

%rip

0x400550

```
00000000000400550 <mult2>:
```

```
400550: mov    %rdi, %rax ←
```

•
•

```
400557: retq
```

控制流示例 #3

```
00000000000400540 <multstore>:
```

•
•
•
•
•

```
400544: callq 400550 <mult2>
```

```
400549: mov    %rax, (%rbx) ←
```

0x130

0x128

0x120

0x118

0x400549

%rsp

0x118

%rip

0x400557

```
00000000000400550 <mult2>:
```

```
400550: mov    %rdi, %rax
```

•
•

```
400557: retq ←
```

控制流示例 #4

```
00000000000400540 <multstore>:
```

•
•
•
•
•

```
400544: callq 400550 <mult2>
```

```
400549: mov    %rax, (%rbx)
```

```
00000000000400550 <mult2>:
```

```
400550: mov    %rdi, %rax
```

•
•

```
400557: retq
```

0x130

0x128

0x120

%rsp

0x120

%rip

0x400549

本节内容

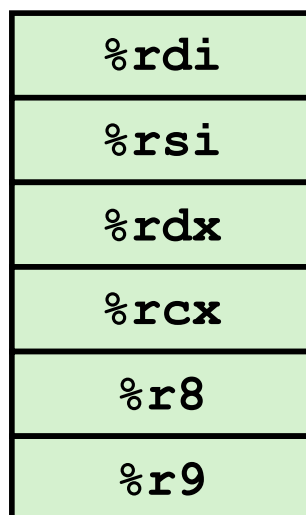
■ 过程

- 栈的结构
- 调用的惯例
 - 控制转移
 - 数据传送
 - 管理局部数据
- 递归

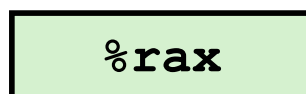
过程调用数据流

寄存器

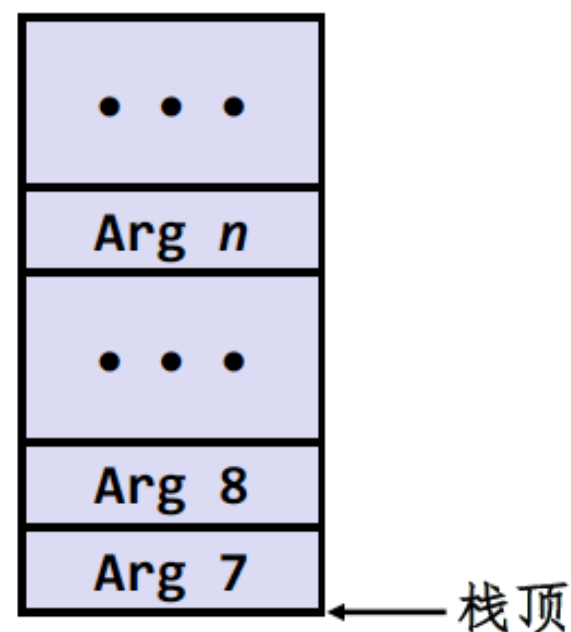
■ 前6个变量



■ 返回值



栈



- 通过栈传递参数时，每个数据的大小都会向上取整到8的倍数
- 只有当需要时才分配栈的空间

过程调用数据流

寄存器

■ 前6个变量

<code>%rdi</code>
<code>%rsi</code>
<code>%rdx</code>
<code>%rcx</code>
<code>%r8</code>
<code>%r9</code>

Operand size (bits)	Argument number					
	1	2	3	4	5	6
64	<code>%rdi</code>	<code>%rsi</code>	<code>%rdx</code>	<code>%rcx</code>	<code>%r8</code>	<code>%r9</code>
32	<code>%edi</code>	<code>%esi</code>	<code>%edx</code>	<code>%ecx</code>	<code>%r8d</code>	<code>%r9d</code>
16	<code>%di</code>	<code>%si</code>	<code>%dx</code>	<code>%cx</code>	<code>%r8w</code>	<code>%r9w</code>
8	<code>%dil</code>	<code>%sil</code>	<code>%dl</code>	<code>%cl</code>	<code>%r8b</code>	<code>%r9b</code>

■ 返回值

<code>%rax</code>

数据流

示例 1

```
void multstore
(long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

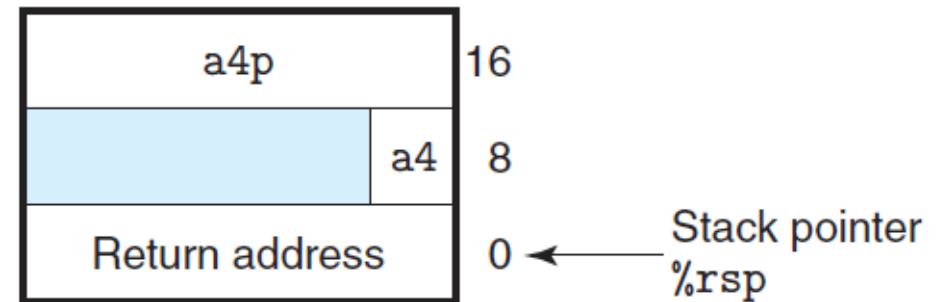
```
0000000000400540 <multstore>:
    # x in %rdi, y in %rsi, dest in %rdx
    . . .
400541: mov     %rdx,%rbx        # Save dest
400544: callq   400550 <mult2>    # mult2(x,y)
    # t in %rax
400549: mov     %rax, (%rbx)      # Save at dest
    . . .
```

```
long mult2
(long a, long b)
{
    long s = a * b;
    return s;
}
```

```
0000000000400550 <mult2>:
    # a in %rdi, b in %rsi
400550: mov     %rdi,%rax        # a
400553: imul    %rsi,%rax        # a * b
    # s in %rax
400557: retq                      # Return
```


数据流示例 2

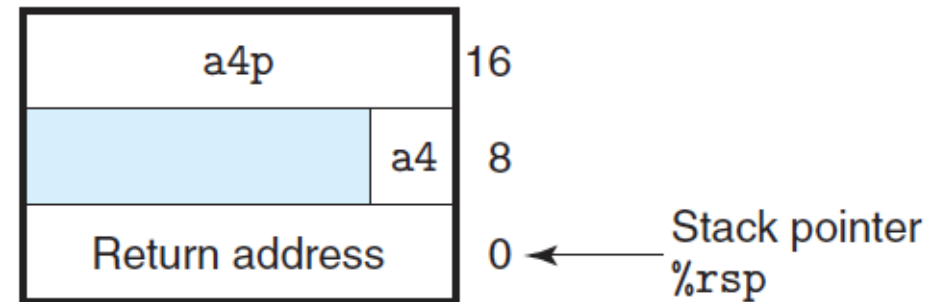
```
void proc(long a1,  
long *a1p,  
int a2, int *a2p,  
short a3, short *a3p,  
char a4, char *a4p)  
{  
    *a1p += a1;  
    *a2p += a2;  
    *a3p += a3;  
    *a4p += a4;  
}
```



```
proc:  
    movq 16(%rsp), %rax    # Fetch a4p(64 bits)  
    addq %rdi, (%rsi)      # *a1p += a1 (64 bits)  
    addl %edx, (%rcx)      # *a2p += a2 (32 bits)  
    addw %r8w, (%r9)       # *a3p += a3 (16 bits)  
    movl 8(%rsp), %edx      # Fetch a4 ( 8 bits)  
    addb %dl, (%rax)        # *a4p += a4 ( 8 bits)  
    ret
```

数据流示例 2

```
void proc(long a1,  
long *a1p,  
int a2, int *a2p,  
short a3, short *a3p,  
char a4, char *a4p)  
{  
    *a1p += a1;  
    *a2p += a2;  
    *a3p += a3;  
    *a4p += a4;  
}
```



```
proc:  
    movq 16(%rsp), %rax    # Fetch a4p(64 bits)  
    addq %rdi, (%rsi)      # *a1p += a1 (64 bits)  
    addl %edx, (%rcx)      # *a2p += a2 (32 bits)  
    addw %r8w, (%r9)       # *a3p += a3 (16 bits)  
    movl 8(%rsp), %edx     # Fetch a4 ( 8 bits)  
    addb %dl, (%rax)       # *a4p += a4 ( 8 bits)  
    ret
```

练习1

```
int procprobl(??, ??, ??, ??)
{
    *u += a;
    *v += b;
    return sizeof(a) + sizeof(b);
}
```

procprob:

```
movslq %edi, %rdi
addq %rdi, (%rdx)
addb %sil, (%rcx)
movl $6, %eax
ret
```

本节内容

■ 过程

- 栈的结构
- 调用的惯例
 - 控制转移
 - 数据传送
 - 管理局部数据
- 递归

基于栈的语言

■ 支持递归的语言

- 例如：C, Pascal, Java
- 代码必须是“可重入的” (Reentrant)
 - 单个过程同时有多个实例
- 需要空间存储每个实例的状态
 - 参数
 - 局部变量
 - 返回地址

■ 栈规则

- 某个过程的状态只需要保存有限的时间
 - 从被调用到返回
- 被调用者在调用者之前返回

■ 栈以**栈帧**的形式进行分配

- 单个过程实例的状态

斐波那契序列：

```
int fib(int n)
{
    if (n == 1 || n == 2)
        return 1;
    return fib(n-1)+fib(n-2);
}
```

调用链示例

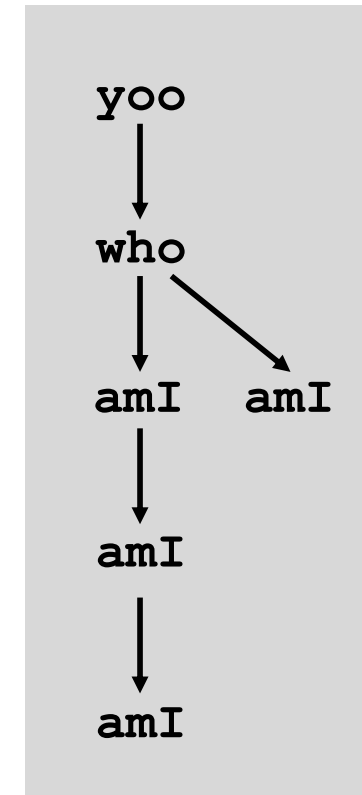
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```

```
who (...)  
{  
  . . .  
  amI ();  
  . . .  
  amI ();  
  . . .  
}
```

```
amI (...)  
{  
  .  
  .  
  amI ();  
  .  
  .  
}
```

程序 `amI ()` 是递归的

调用链示例



栈帧

■ 内容

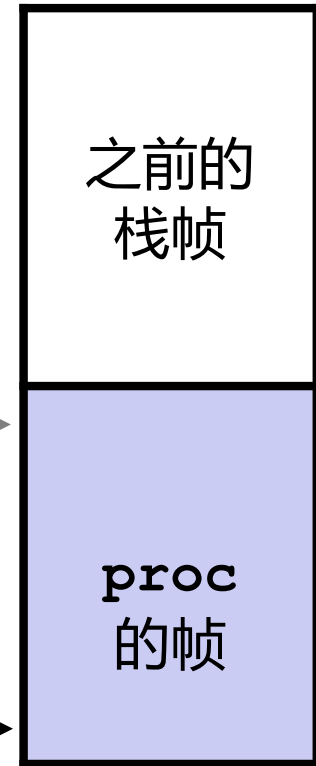
- 返回信息
- 局部存储（如果需要）
- 临时空间（如果需要）

■ 管理

- 进入过程时分配的空间
 - “准备工作” 代码
 - 包括 `call` 指令压入的数据
- 返回时释放空间
 - “结束工作” (tear-down) 代码
 - 包括 `ret` 指令弹出的数据

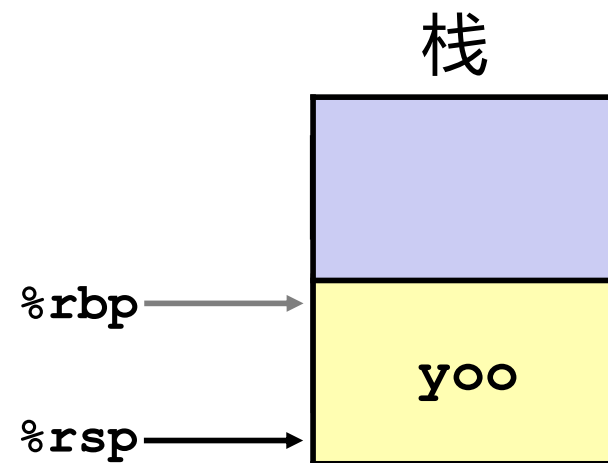
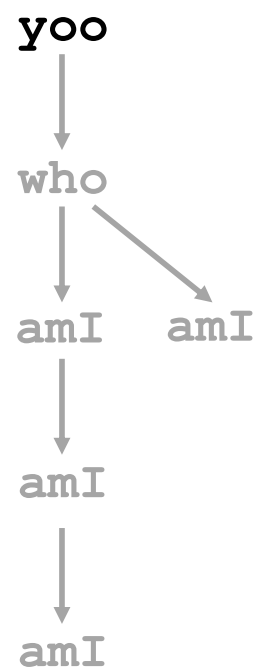
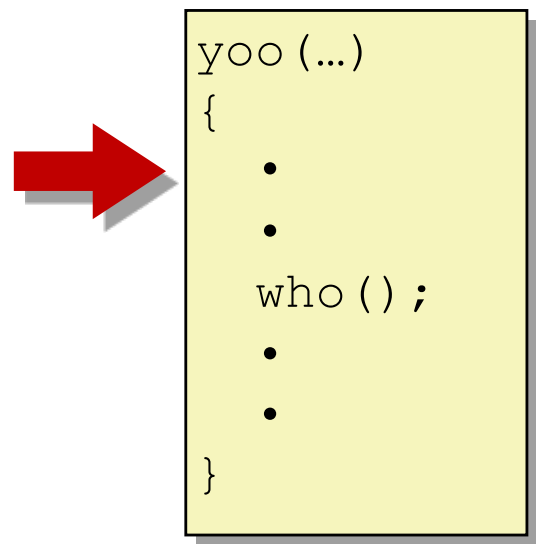
栈帧指针：`%rbp`
(可选)

栈指针：`%rsp`

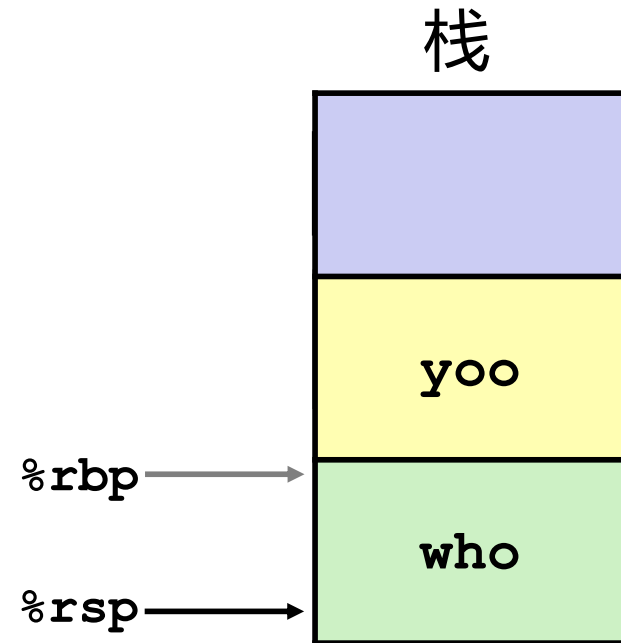
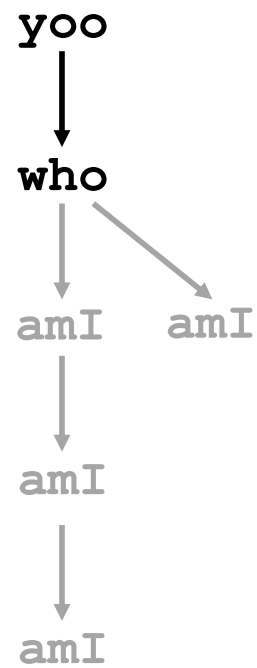
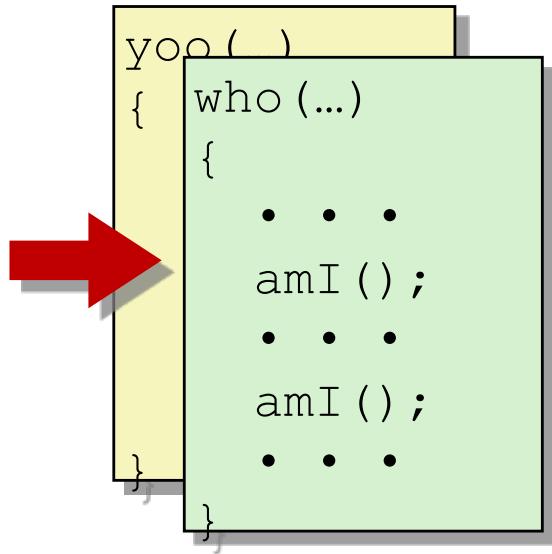


栈“顶”

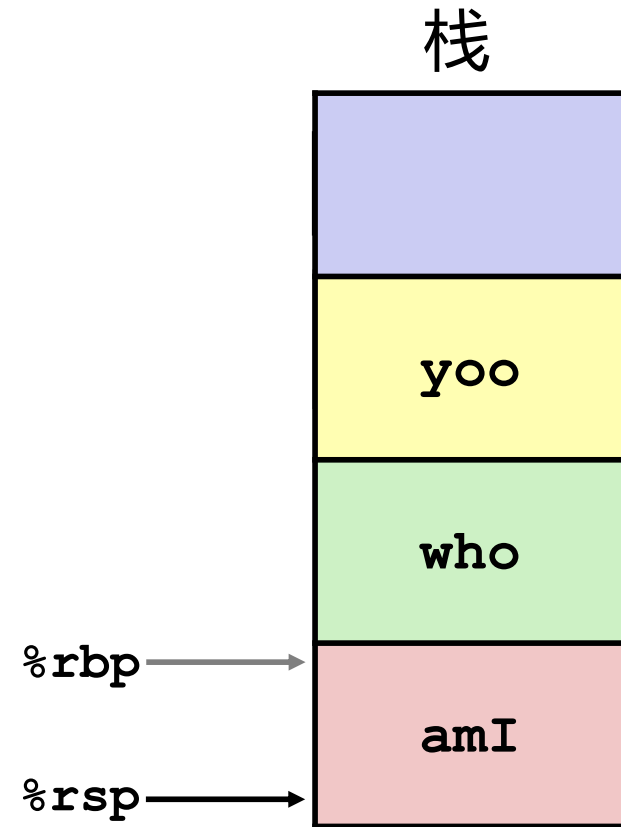
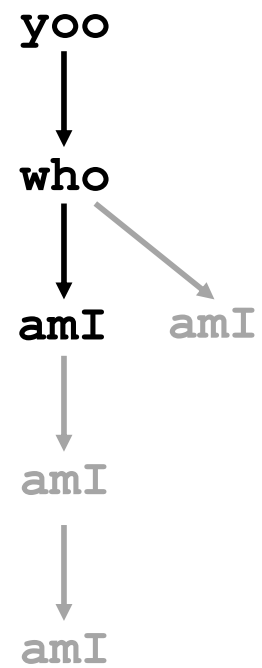
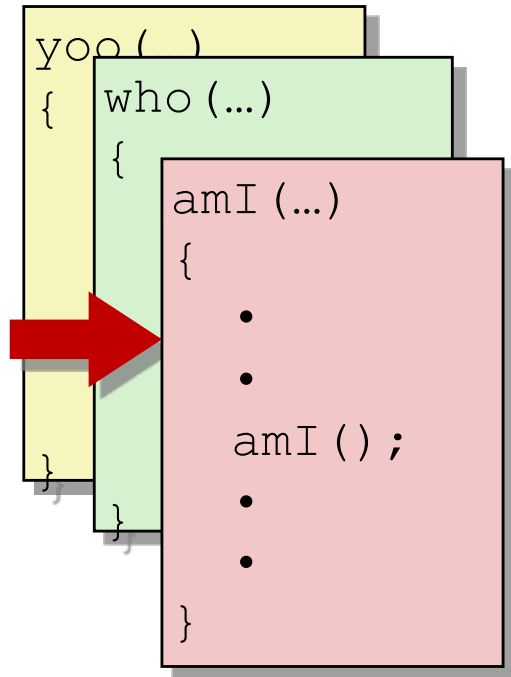
示例



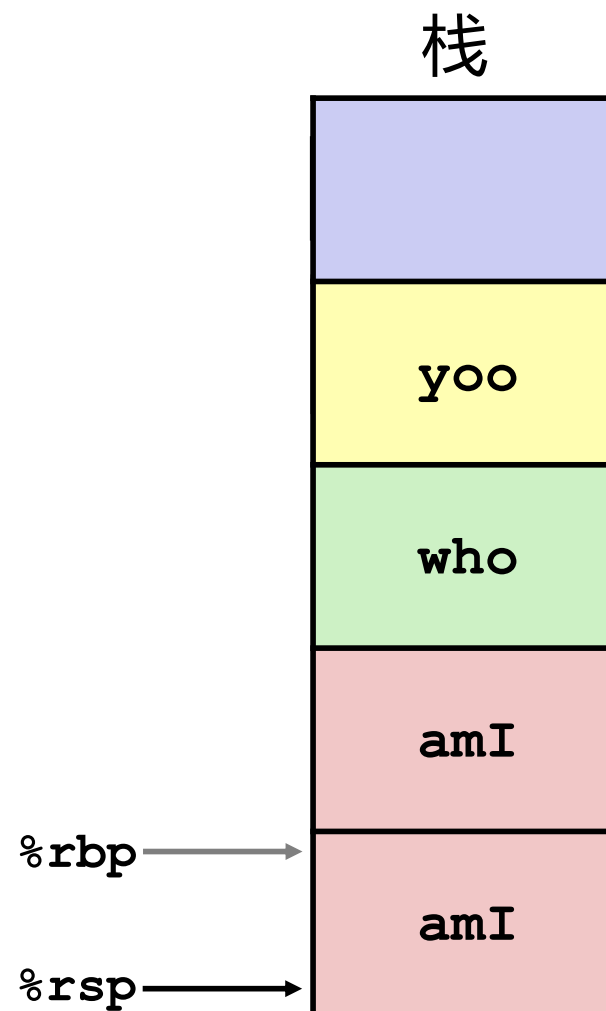
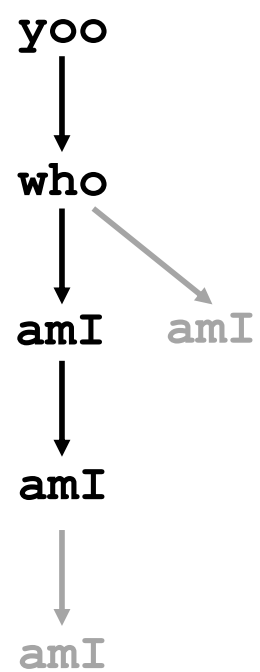
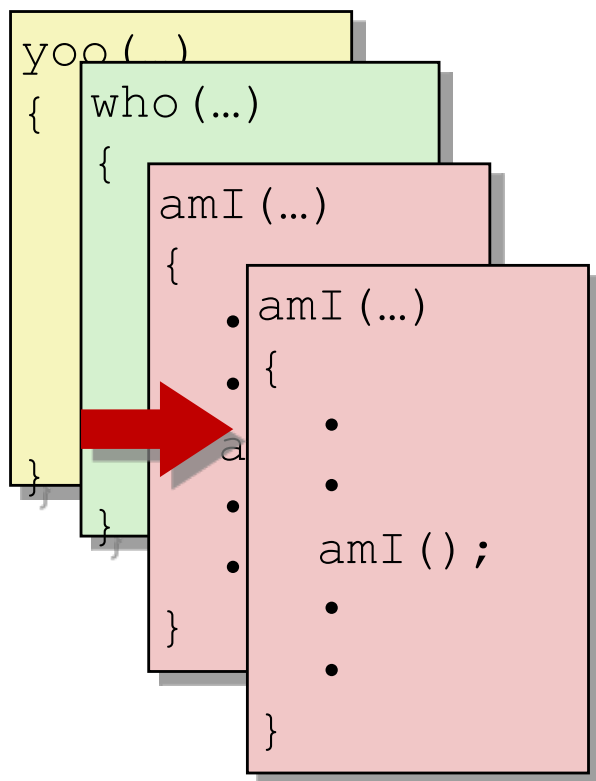
示例



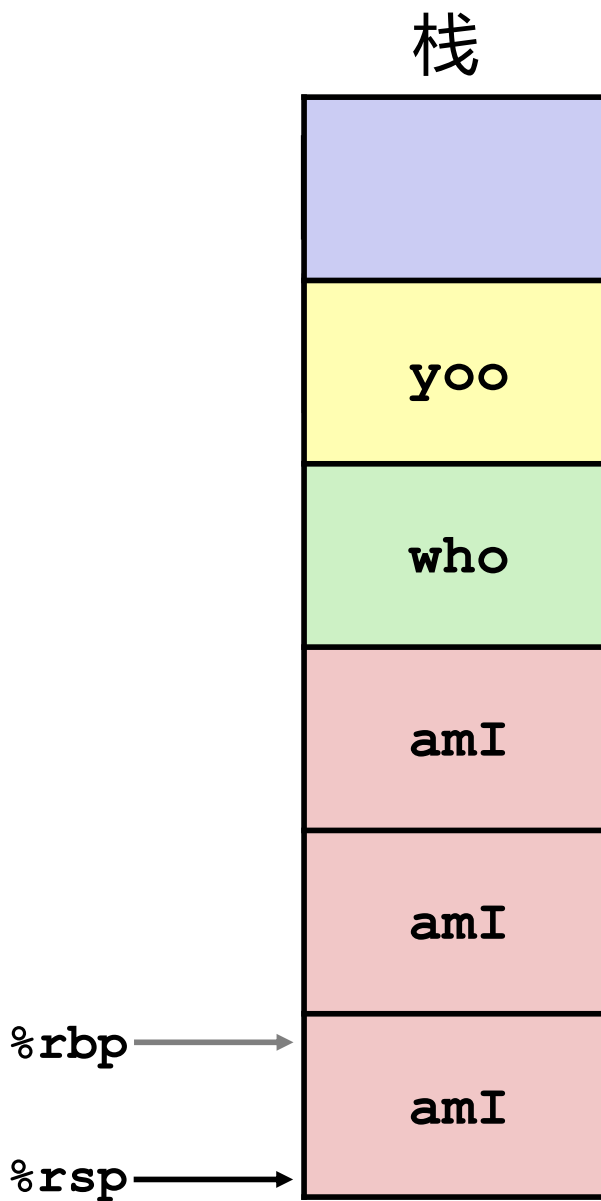
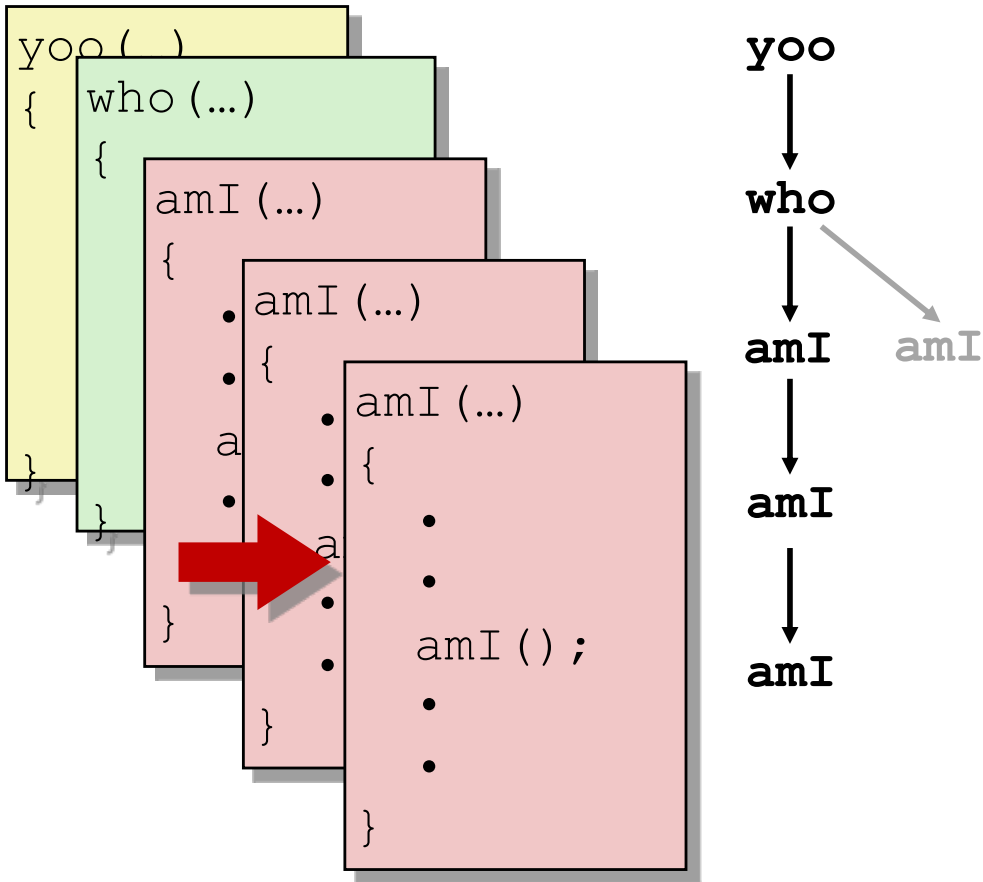
示例



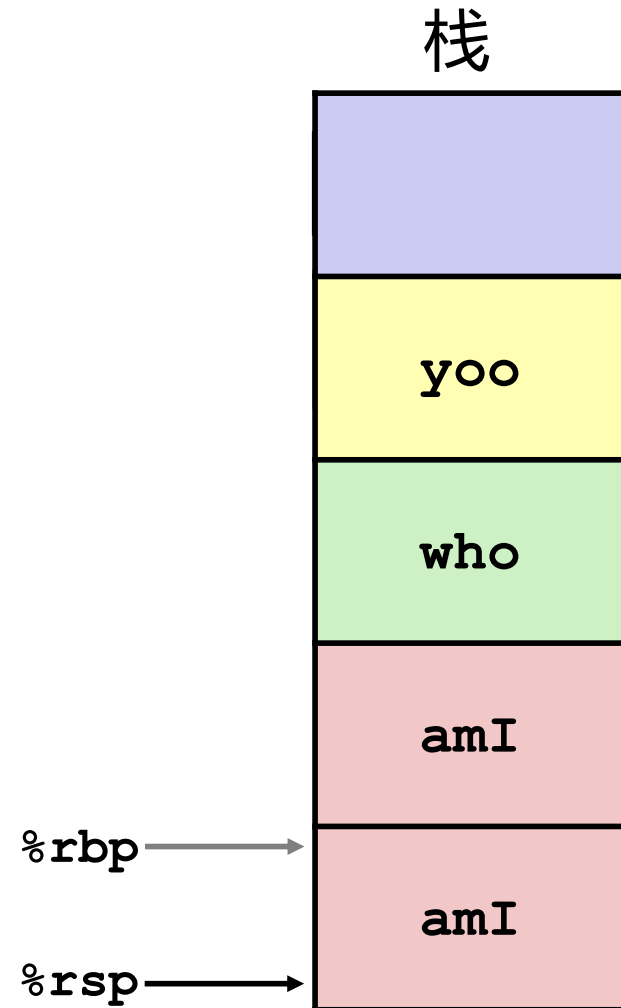
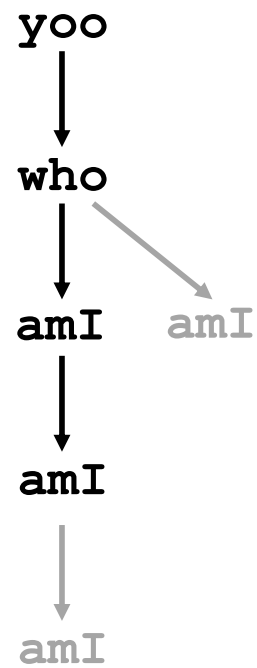
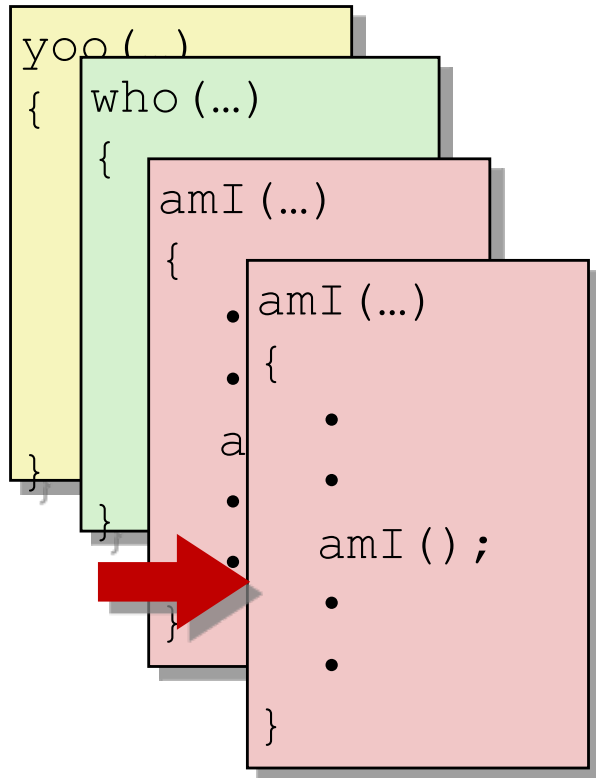
示例



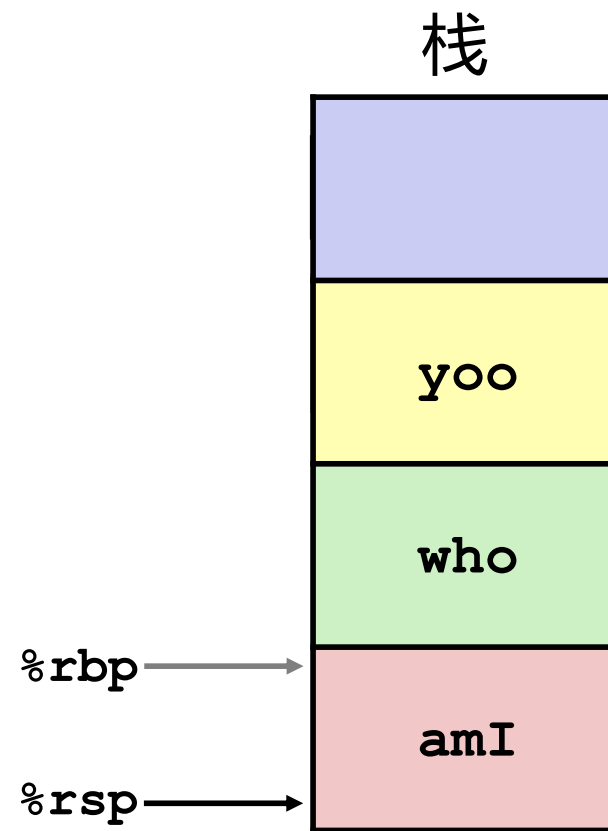
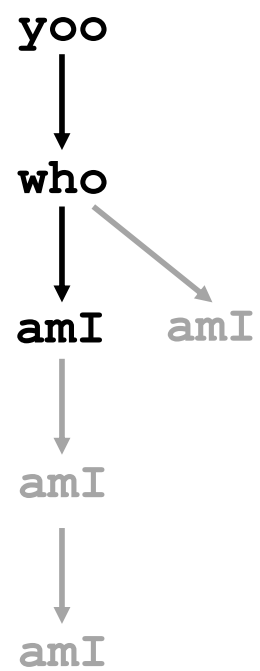
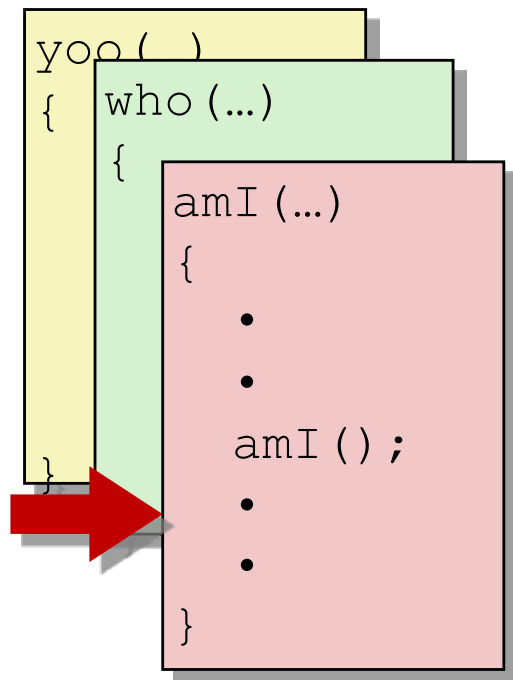
示例



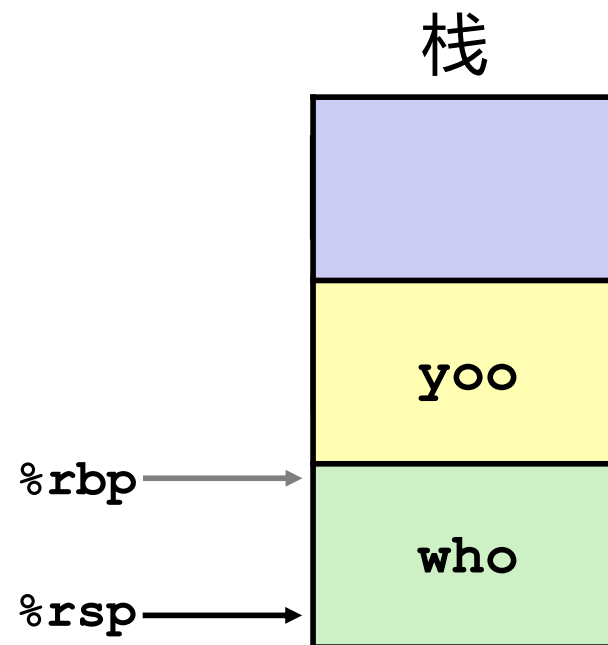
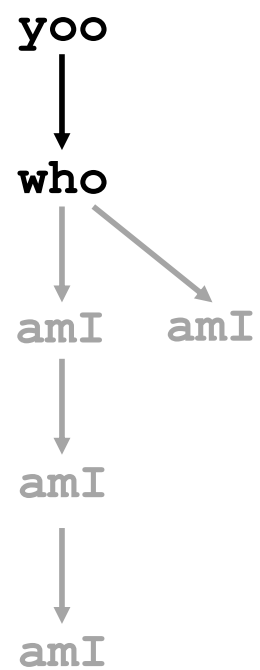
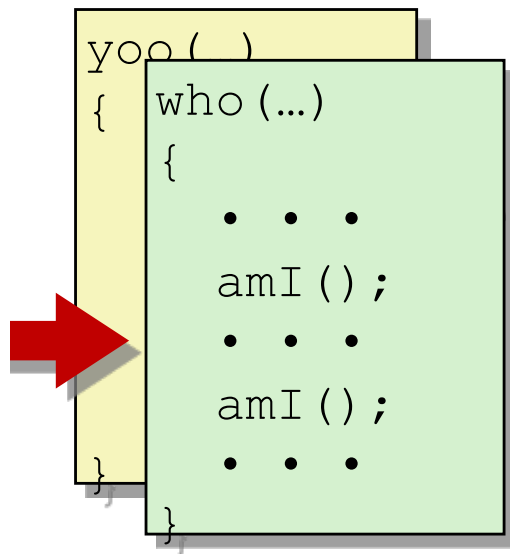
示例



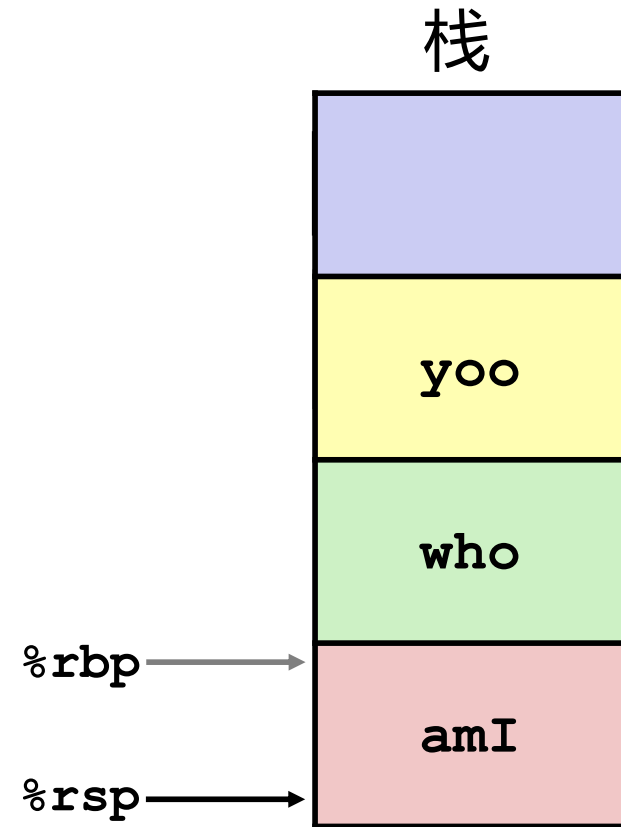
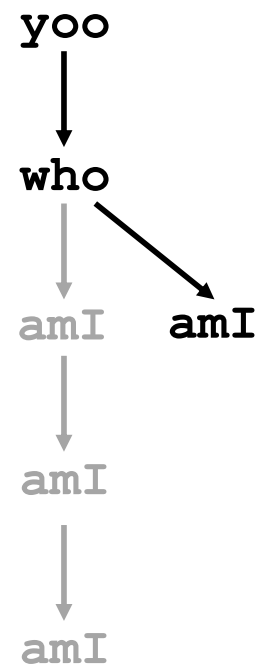
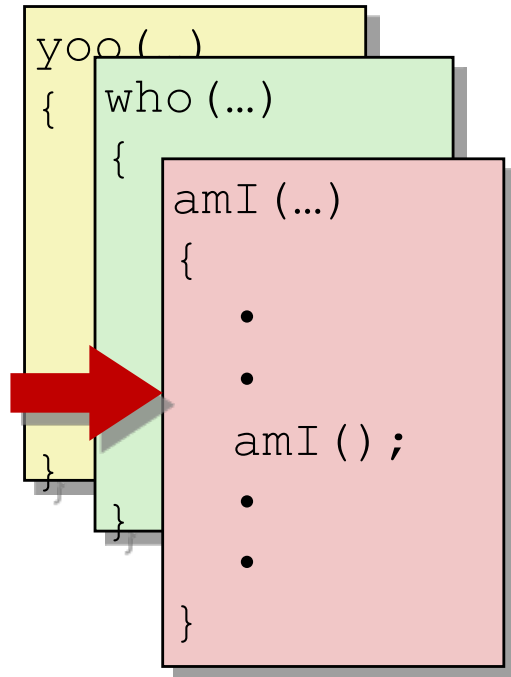
示例



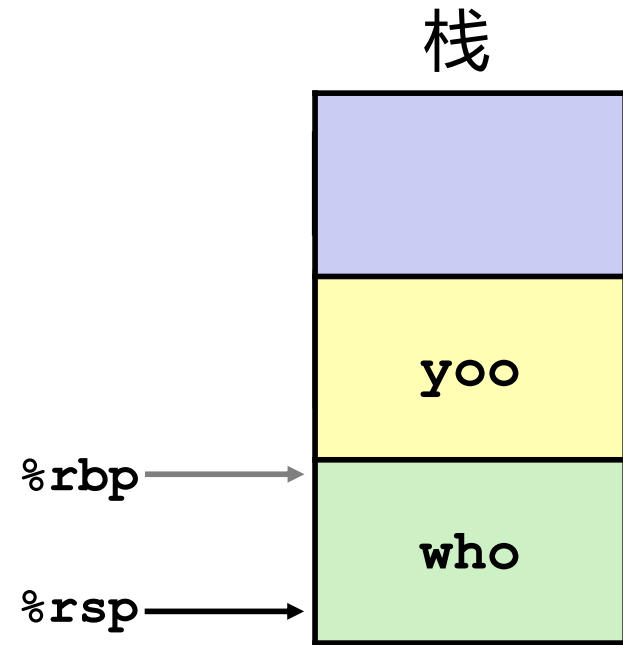
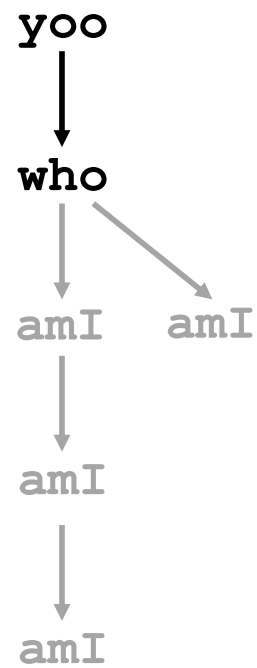
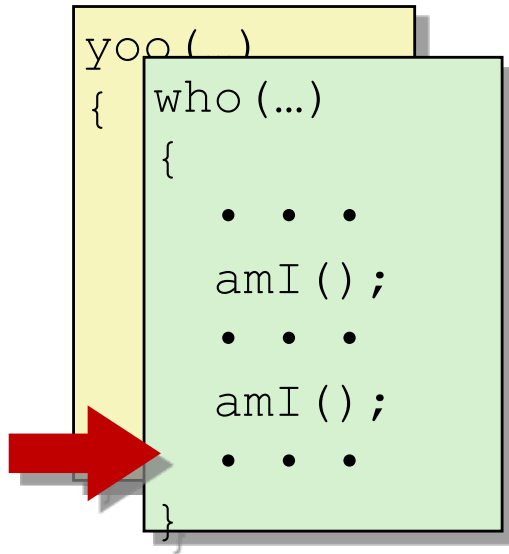
示例



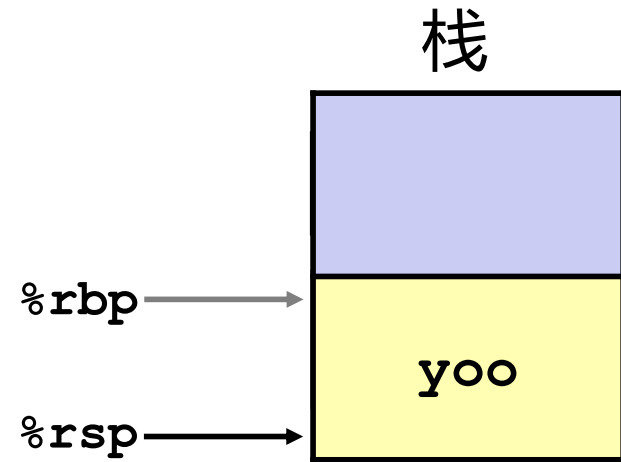
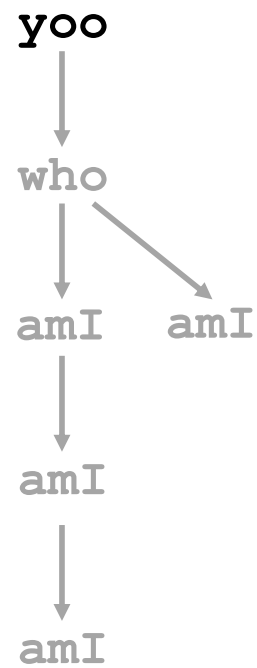
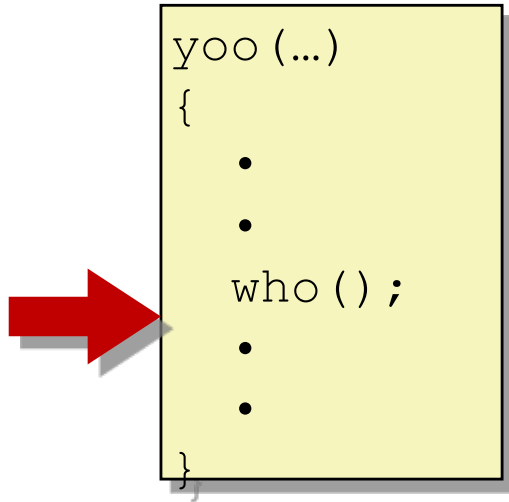
示例



示例



示例



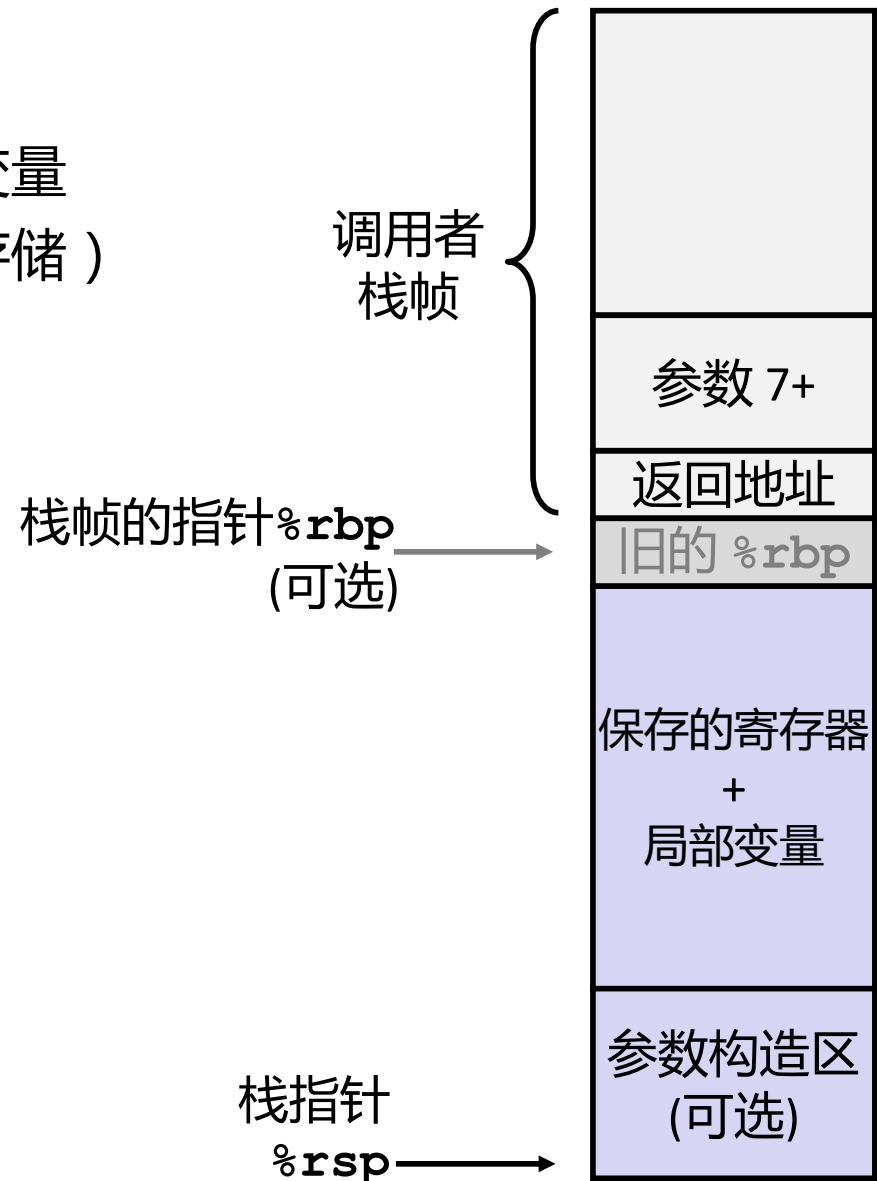
x86-64/Linux 栈帧

■ 当前栈帧（从栈顶到栈底）

- 参数构造区：为调用的函数准备变量
- 局部变量（如果在寄存器中不能存储）
- 保存的寄存器
- 旧栈帧的指针（可选）

■ 调用者的栈帧

- 返回地址
 - 通过 `call` 调用指令压入栈
- 为调用函数准备的参数



示例：incr

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

示例：incr

```
long incr(long *p, long val) {  
    long x = *p;  
    long y = x + val;  
    *p = y;  
    return x;  
}
```

```
incr:  
    movq    (%rdi), %rax  
    addq    %rax, %rsi  
    movq    %rsi, (%rdi)  
    ret
```

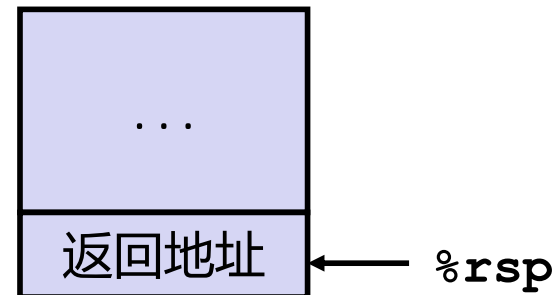
寄存器	值
%rdi	变量 p
%rsi	变量 val, y
%rax	返回值 x

示例：调用 `incr` #1

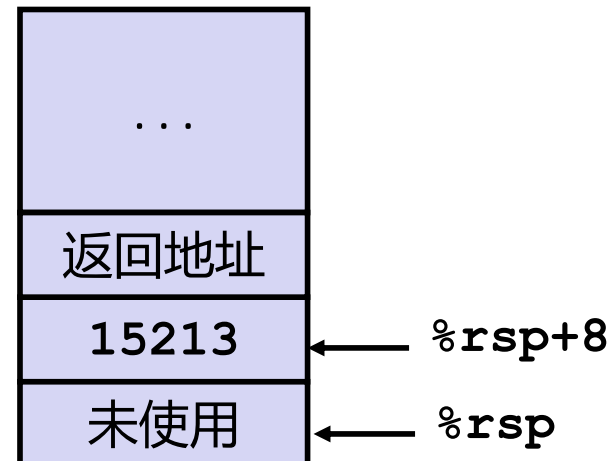
```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

初始的栈结构



现在的栈结构

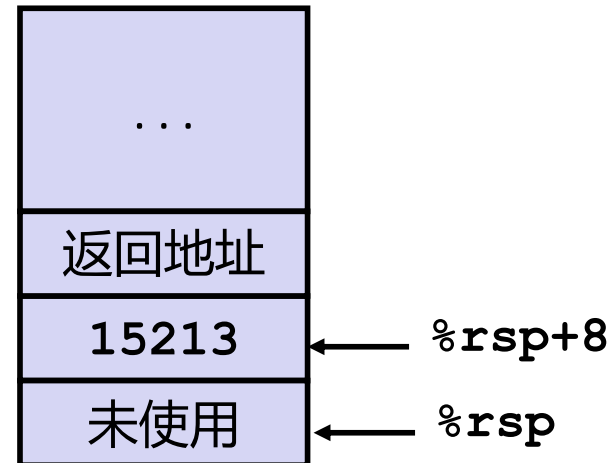


示例：调用 `incr` #2

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

栈的结构

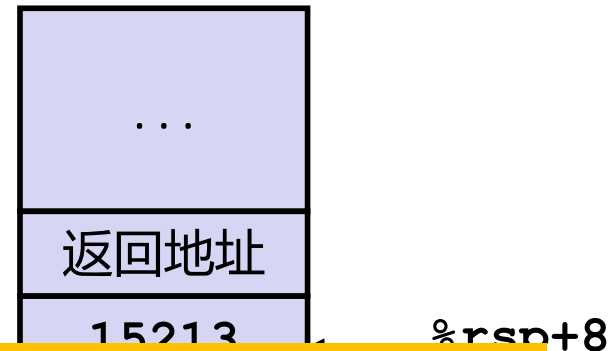


寄存器	值
<code>%rdi</code>	<code>&v1</code>
<code>%rsi</code>	3000

示例：调用 `incr #2`

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

栈的结构



Aside 1: `movl $3000, %esi`

- 注意：`movl -> %eax` 将高位 32 位置零。
- 为什么使用 `movl` 而不是 `movq`？短了 1 个字节。

```
leaq    8(%rsp), %rdi  
call    incr  
addq    8(%rsp), %rax  
addq    $16, %rsp  
ret
```

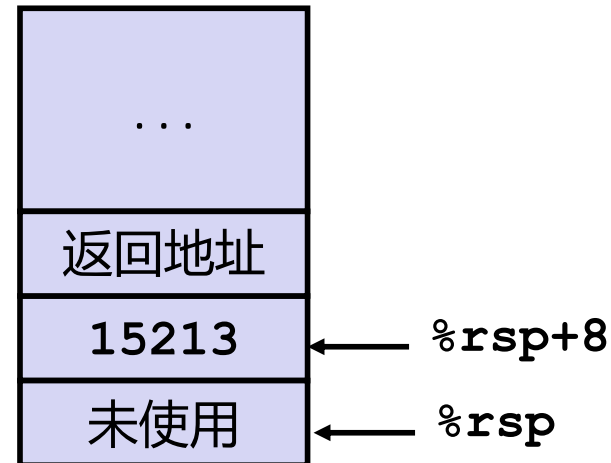
...	...
%rsi	3000

示例：调用 `incr` #2

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi
```

栈的结构



寄存器	值
%rdi	&v1
%rsi	3000

Aside 2: `leaq 8(%rsp), %rdi`

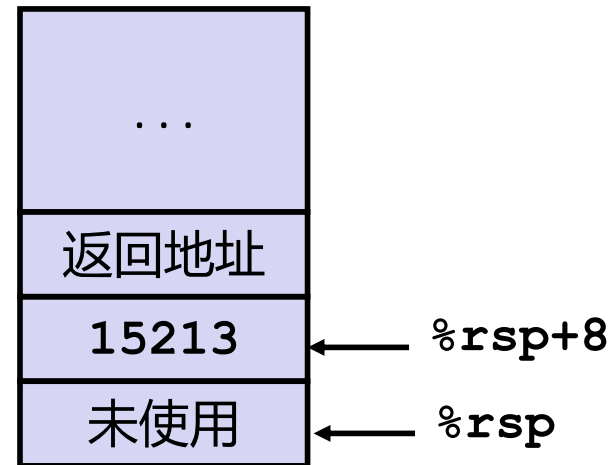
- 计算 `%rsp+8`
- 实际上 `leaq` 的本意是用来计算地址的！

示例：调用 `incr` #2

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

栈的结构



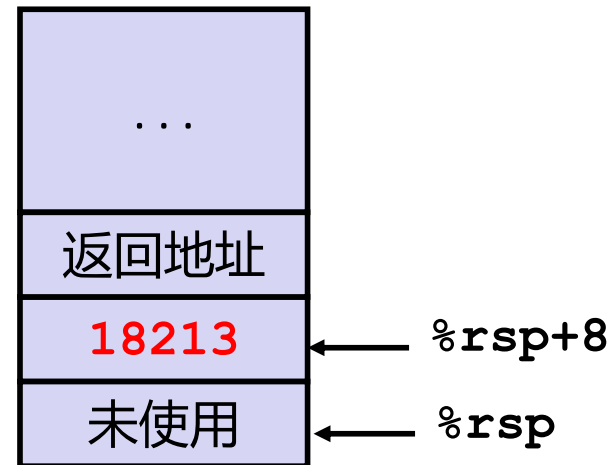
寄存器	值
<code>%rdi</code>	<code>&v1</code>
<code>%rsi</code>	3000

示例：调用 `incr` #3

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

栈的结构



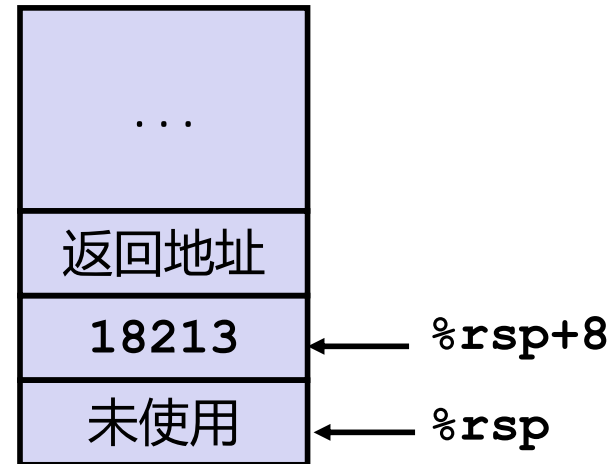
寄存器	值
%rdi	&v1
%rsi	3000

示例：调用 `incr` #4

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

栈的结构



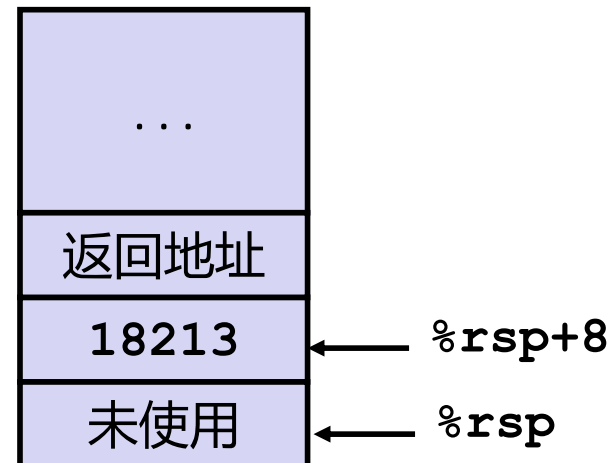
寄存器	值
<code>%rax</code>	返回值

示例：调用 `incr` #5a

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

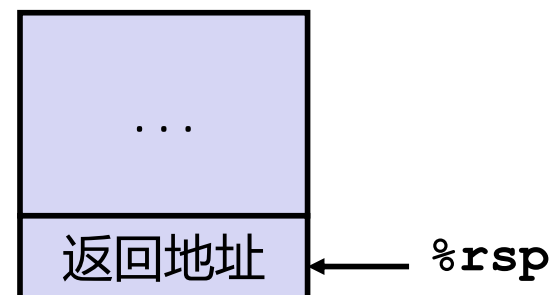
```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

栈的结构



寄存器	值
<code>%rax</code>	返回值

更新的栈结构

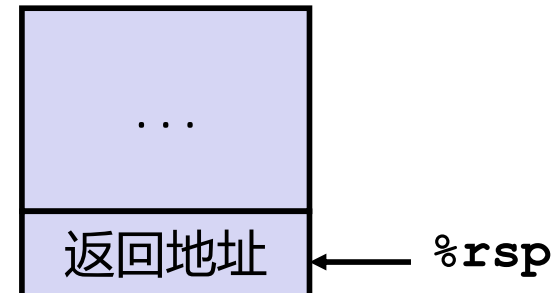


示例：调用 `incr #5b`

```
long call_incr() {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return v1+v2;  
}
```

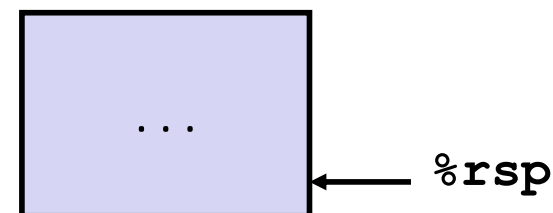
```
call_incr:  
    subq    $16, %rsp  
    movq    $15213, 8(%rsp)  
    movl    $3000, %esi  
    leaq    8(%rsp), %rdi  
    call    incr  
    addq    8(%rsp), %rax  
    addq    $16, %rsp  
    ret
```

更新后的栈结构



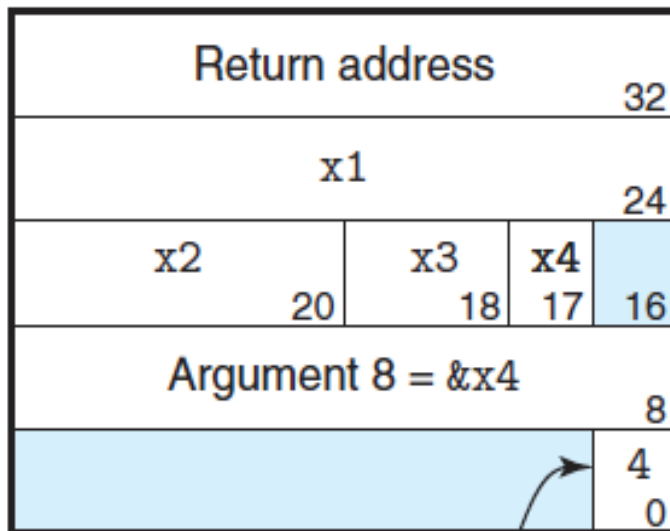
寄存器	值
<code>%rax</code>	返回值

最后的栈结构



其他示例

```
long call_proc()
{
    long x1 = 1; int x2 = 2;
    short x3 = 3; char x4 = 4;
    proc(x1, &x1, x2, &x2, x3, &x3,
        x4, &x4);
    return (x1+x2)*(x3-x4);
}
```



Argument 7

```
call_proc:
    subq $32, %rsp           #Allocate 32-byte stack frame
    movq $1, 24(%rsp)        #Store 1 in &x1
    movl $2, 20(%rsp)        #Store 2 in &x2
    movw $3, 18(%rsp)        #Store 3 in &x3
    movb $4, 17(%rsp)        #Store 4 in &x4
    leaq 17(%rsp), %rax       #Create &x4
    movq %rax, 8(%rsp)        #Store &x4 as argument 8
    movl $4, (%rsp)           #Store 4 as argument 7
    leaq 18(%rsp), %r9        #Pass &x3 as argument 6
    movl $3, %r8d             #Pass 3 as argument 5
    leaq 20(%rsp), %rcx       #Pass &x2 as argument 4
    movl $2, %edx             #Pass 2 as argument 3
    leaq 24(%rsp), %rsi       #Pass &x1 as argument 2
    movl $1, %edi             #Pass 1 as argument 1
    call proc
    movslq 20(%rsp), %rdx     #Get x2 and convert to long
    addq 24(%rsp), %rdx       #Compute x1+x2
    movswl 18(%rsp), %eax     #Get x3 and convert to int
    movsbl 17(%rsp), %ecx     #Get x4 and convert to int
    subl %ecx, %eax           #Compute x3-x4
    cltq                      #Convert to long
    imulq %rdx, %rax          #Compute (x1+x2) * (x3-x4)
    addq $32, %rsp           #Deallocate stack frame
    ret
```

Stack pointer
%rsp

寄存器使用惯例

■ 当程序 yoo 调用 who 时：

- yoo 是调用者 (caller)
- who 是被调用者 (callee)

■ 寄存器可以用于临时存储吗？

```
yoo:
    . . .
    movq $15213, %rdx
    call who
    addq %rdx, %rax
    . . .
    ret
```

```
who:
    . . .
    subq $18213, %rdx
    . . .
    ret
```

- 寄存器 %rdx 的内容会被 who 覆盖
- 如果被调用者破坏了你的寄存器，它的价值就会丢失！
 - 需要调用者/被调用者之间的协调

寄存器使用惯例

■ 当程序 yoo 调用 who 时：

- yoo 是调用者 (caller)
- who 是被调用者 (callee)

■ 寄存器可以用于临时存储吗？

■ 约定

- “调用者保存”
 - 函数调用前，调用者要在自己的栈帧里保存临时值
- “被调用者保存”
 - 被调用者在使用寄存器值之前，在自己的栈帧中存储这些临时值
 - 被调用者在返回调用者之前，恢复这些值

x86-64 Linux 寄存器的使用 #1

■ **%rax**

- 返回值
- **调用者保存**
- 程序可以修改

■ **%rdi, ..., %r9**

- 参数
- **调用者保存**
- 程序可以修改

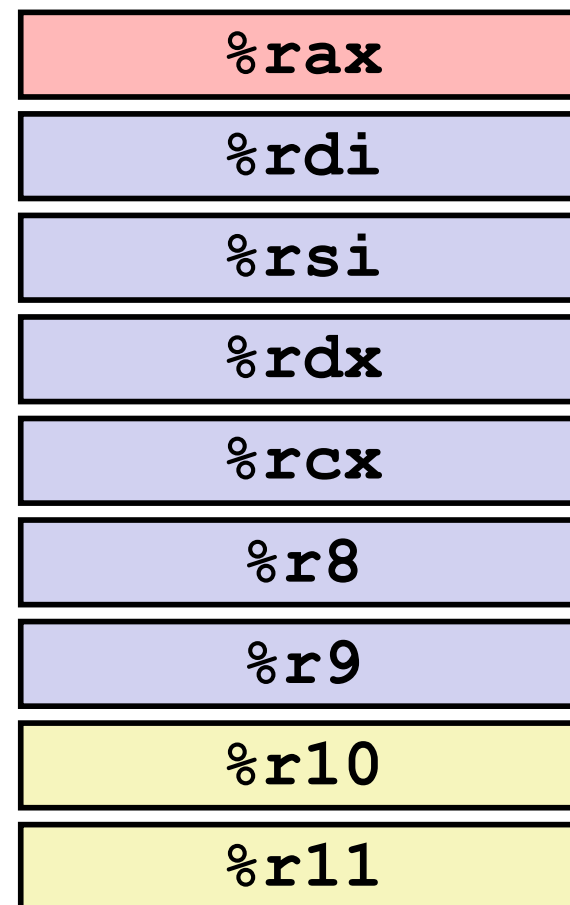
■ **%r10, %r11**

- **调用者保存**
- 程序可以修改

调用者保存
返回值

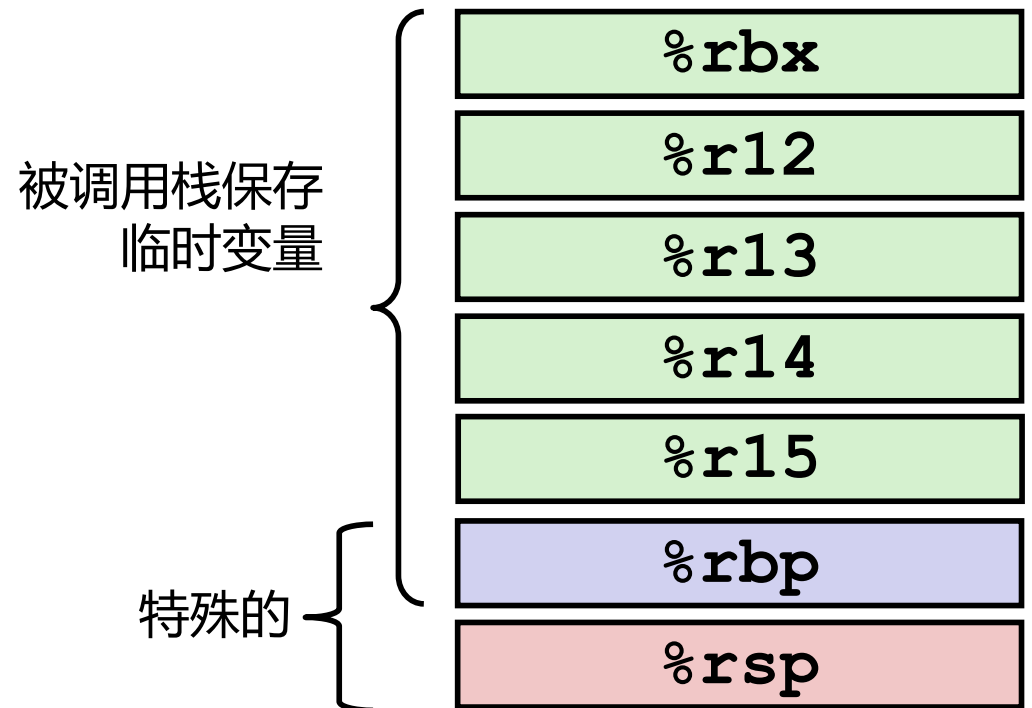
调用者保存
参数

调用者保存
临时变量



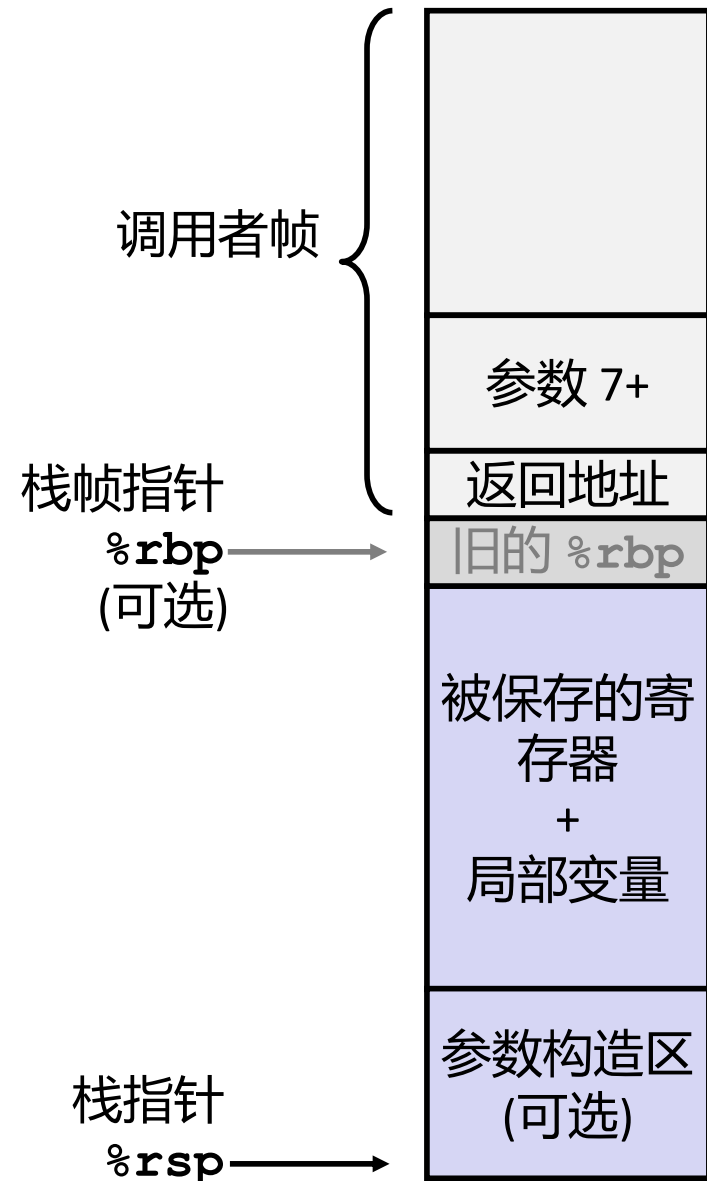
x86-64 Linux寄存器的使用 #2

- `%rbx`, `%r12`, `%r13`,
`%r14`, `%r15`
 - **被调用者保存**
 - 被调用者必须保存并恢复值
- `%rbp`
 - **被调用者保存**
 - 被调用者必须保存并恢复值
 - 可用被用作栈帧指针
- `%rsp`
 - 一种特殊的**被调用者保存**寄存器
 - 在退出过程时恢复成原来的值



x86-64 Linux寄存器的使用 #2

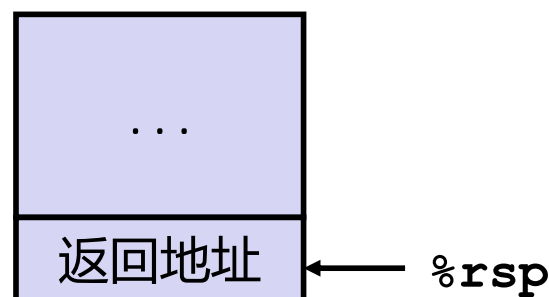
- `%rbx`, `%r12`, `%r13`,
`%r14`, `%r15`
 - **被调用者保存**
 - 被调用者必须保存并恢复值
- `%rbp`
 - **被调用者保存**
 - 被调用者必须保存并恢复值
 - 可用被用作栈帧指针
- `%rsp`
 - 一种特殊的**被调用者保存**寄存器
 - 在退出过程时恢复成原来的值



被调用者保存寄存器示例 #1

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

初始栈结构



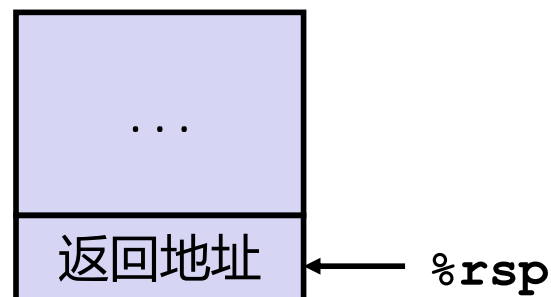
- **x** 进入寄存器 `%rdi`。
- 我们需要 `%rdi` 来调用 `incr`。
- 为了在调用 `incr` 后使用它，**x** 应该放在哪里？

被调用者保存寄存器示例 #2

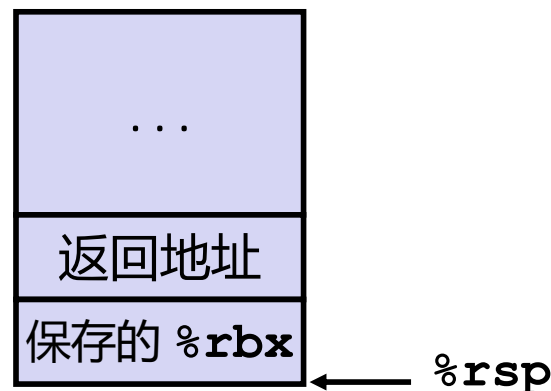
```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

初始栈结构



当前栈结构

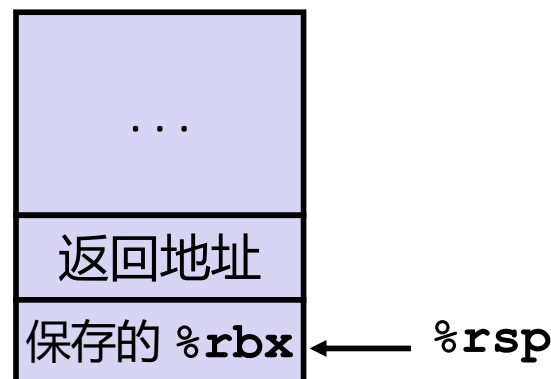


被调用者保存寄存器示例 #3

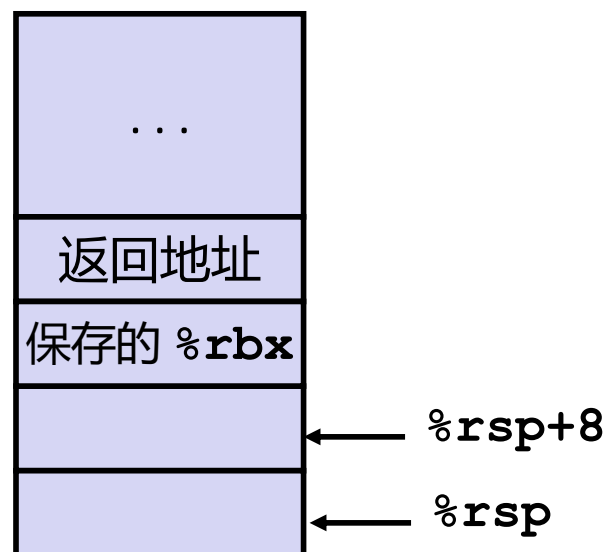
```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

栈结构



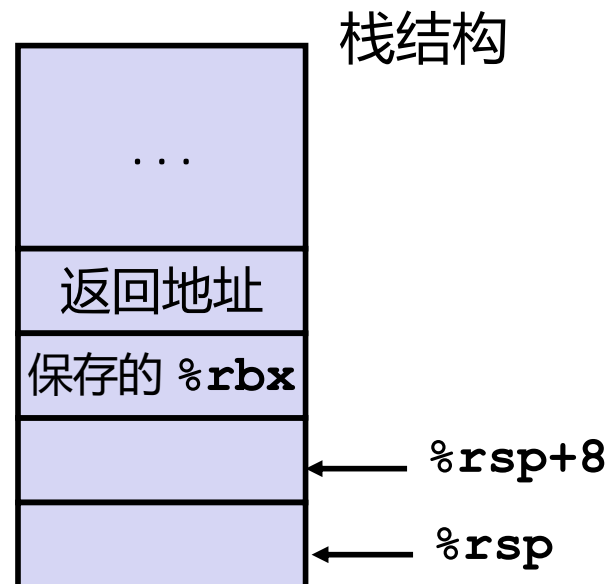
结果的栈结构



被调用者保存寄存器示例 #4

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



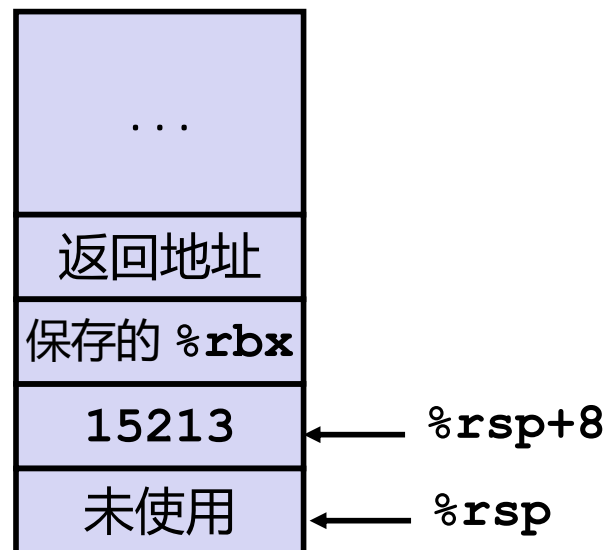
- **x** 保存在 **%rbx** 中, 一个被调用者保存寄存器

被调用者保存寄存器示例 #5

栈结构

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



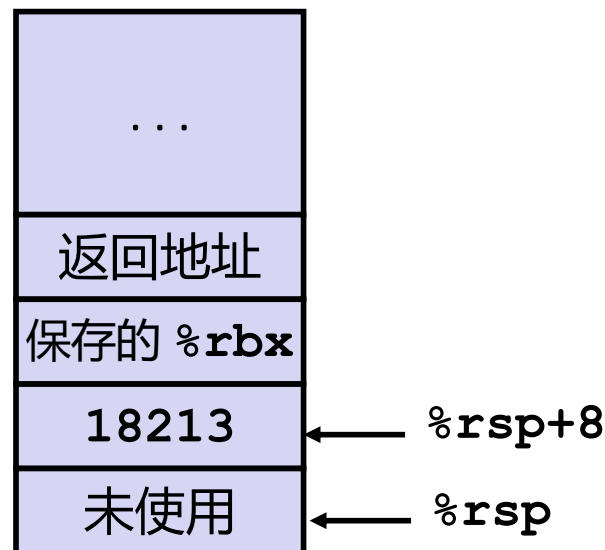
- **x** 保存在 **%rbx** 中，
一个被调用者保存寄存器

被调用者保存寄存器示例 #6

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

栈结构



从 incr 返回后：

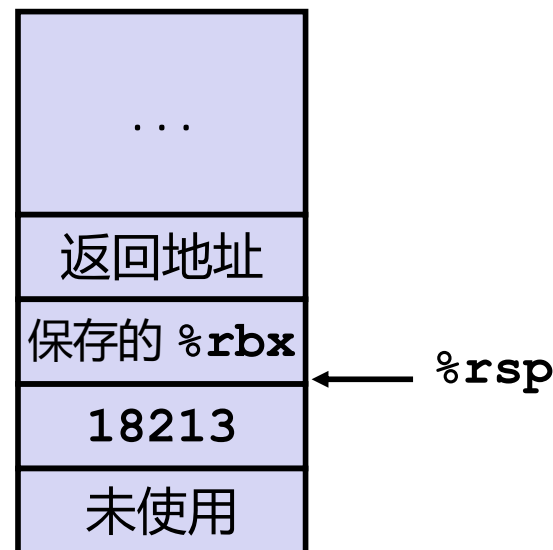
- **x** 保存在 **%rbx**
- 在 **%rax** 中返回 **val**和**v2**
- 计算 **x+v2**: **addq %rbx,%rax**

被调用者保存寄存器示例 #7

栈结构

```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```



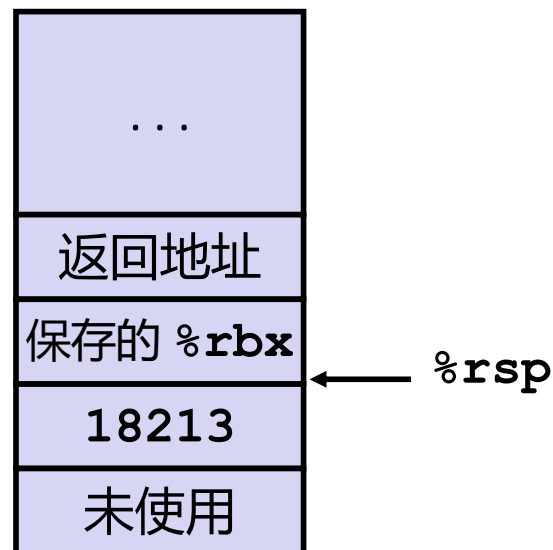
- 返回结果于 **%rax**

被调用者保存寄存器示例 #8

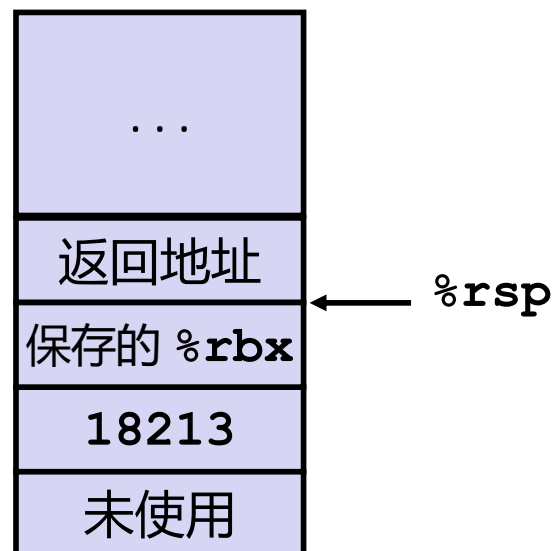
```
long call_incr2(long x) {  
    long v1 = 15213;  
    long v2 = incr(&v1, 3000);  
    return x+v2;  
}
```

```
call_incr2:  
    pushq    %rbx  
    subq     $16, %rsp  
    movq     %rdi, %rbx  
    movq     $15213, 8(%rsp)  
    movl     $3000, %esi  
    leaq     8(%rsp), %rdi  
    call     incr  
    addq     %rbx, %rax  
    addq     $16, %rsp  
    popq     %rbx  
    ret
```

初始栈结构



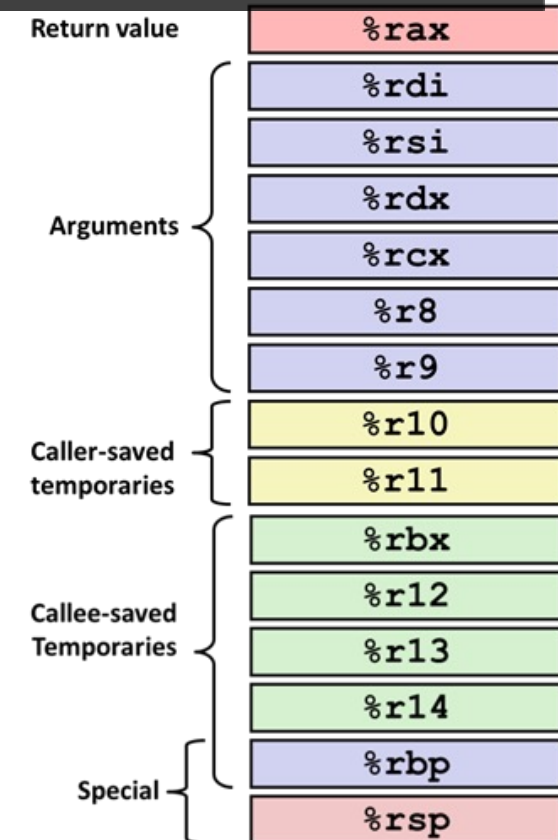
最终栈结构



课堂练习#2

```
long add5(long b0, long b1, long b2, long b3, long b4) {  
    return b0+b1+b2+b3+b4;  
}  
  
long add10(long a0, long a1, long a2, long a3, long a4, long a5,  
           long a6, long a7, long a8, long a9) {  
    return add5(a0, a1, a2, a3, a4)+  
           add5(a5, a6, a7, a8, a9);  
}
```

- Where are a0,..., a9 passed?
rdi, rsi, rdx, rcx, r8, r9, stack
- Where are b0,..., b4 passed?
rdi, rsi, rdx, rcx, r8
- Which registers do we need to save?
Ill-posed question. Need assembly.



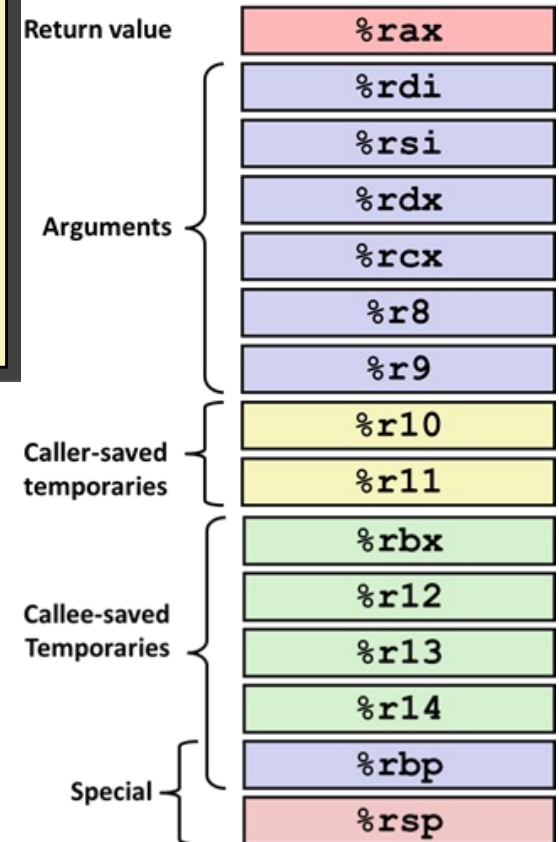
课堂练习#3

```
long add5(long b0, long b1, long b2, long b3, long b4) {
    return b0+b1+b2+b3+b4;
}

long add10(long a0, long a1, long a2, long a3, long a4, long a5,
long a6, long a7, long a8, long a9) {
    return add5(a0, a1, a2, a3, a4)+
        add5(a5, a6, a7, a8, a9);
}
```

```
add10:
    pushq    %rbp
    pushq    %rbx
    movq     %r9, %rbp
    call     add5
    movq     %rax, %rbx
    movq     48(%rsp), %r8
    movq     40(%rsp), %rcx
    movq     32(%rsp), %rdx
    movq     24(%rsp), %rsi
    movq     %rbp, %rdi
    call     add5
    addq     %rbx, %rax
    popq     %rbx
    popq     %rbp
    ret
```

```
add5:
    addq     %rsi, %rdi
    addq     %rdi, %rdx
    addq     %rdx, %rcx
    leaq     (%rcx,%r8), %rax
    ret
```



本节内容

■ 过程

- 栈的结构
- 调用的惯例
 - 控制转移
 - 数据传送
 - 管理局部数据
- 递归

递归函数

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```


递归函数终止情况

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

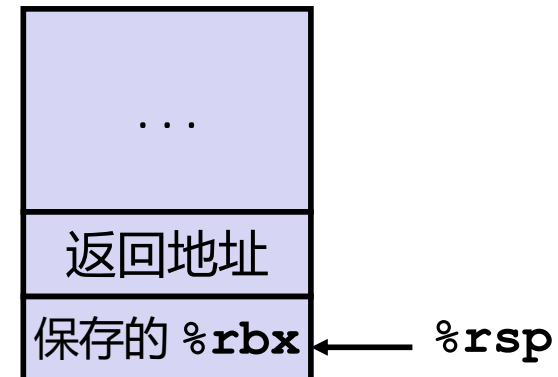
寄存器	值	类型
%rdi	x	参数
%rax	返回值	返回值

递归函数寄存器保存

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

寄存器	值	类型
%rdi	x	参数

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```



递归函数调用准备

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

寄存器	值	类型
%rdi	x >> 1	递归参数
%rbx	x & 1	被调用者保存

递归函数调用

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

寄存器	值	类型
%rbx	x & 1	被调用者保存
%rax	递归调用返回值	

递归函数结果

```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
.L6:
    rep; ret
```

寄存器	值	类型
%rbx	x & 1	被调用者保存
%rax	返回值	

递归函数完成

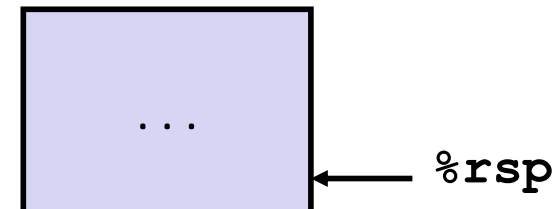
```
/* Recursive popcount */
long pcount_r(unsigned long x) {
    if (x == 0)
        return 0;
    else
        return (x & 1)
            + pcount_r(x >> 1);
}
```

```
pcount_r:
    movl    $0, %eax
    testq   %rdi, %rdi
    je      .L6
    pushq   %rbx
    movq    %rdi, %rbx
    andl    $1, %ebx
    shrq    %rdi
    call    pcount_r
    addq    %rbx, %rax
    popq    %rbx
```

.L6:

rep; ret

寄存器	值	类型
%rax	返回值	返回值



关于递归的观察

■ 对递归不需要特殊处理

- 栈帧意味着每一次函数调用都有私有存储空间
 - 保存的寄存器和局部变量
 - 保存的返回地址
- 寄存器保存惯例阻止一次函数调用篡改另一次调用的数据
 - 除非 C 代码中明确要这么做（例如，后面会讨论的缓冲区溢出）
- 栈规则与过程调用/返回模式保持一致
 - 如果 P 调用 Q，则 Q 在 P 之前返回
 - 后进先出
- 也适用于相互递归
 - P调用Q； Q调用P

x86-64 过程总结

■ 要点

- 栈是过程调用/返回的标准数据结构
 - 如果 P 调用 Q，则 Q 在 P 之前返回

■ 由标准调用惯例处理递归（以及相互递归）

- 可以安全地将值存储在局部栈帧和被调用者保存寄存器中
- 将函数参数放在栈顶
- 结果返回 %rax

■ 指针是值的地址

- 值存储在栈上或为全局变量

